

2025 年全国大学生信息安全竞赛

作品报告

作品名称： 数字遗产管家

电子邮箱： _____

提交日期： _____

摘要

加密货币私钥、云端账户、加密文件和智能合约权限等数字资产已成为新型个人财产。据统计，全球加密货币持有者超 4.2 亿人，因私钥丢失永久无法访问的比特币达 370 余万枚。传统遗嘱、公证和平台申诉机制在数字资产场景中面临困境：私钥无法通过法院强制转移，提前公开凭据带来泄露风险，中心化平台恢复流程周期长且覆盖范围有限。本作品数字遗产管家（Digital Legacy Guardian）围绕数字遗产继承问题，设计并实现了一个基于门限密码学、可验证秘密共享和智能合约的数字资产继承系统。

系统采用 React + TypeScript 前端、Node.js + Express 后端和 Solidity 智能合约三层架构，将数字遗产继承抽象为“资产托管、条件触发、份额验证、密钥恢复、资产释放”五个有序阶段，每个阶段设密码学和状态双重约束。核心安全机制包括：在 secp256k1 标量域上实现 Shamir Secret Sharing，主密钥拆分为 n 个份额，任意 t 个可通过拉格朗日插值恢复，少于 t 个从信息论角度无法推断秘密；引入双多项式 Pedersen 同态承诺结构，实现 server 零信任的可验证份额分发与同态刷新；采用 StoredShare 与 Share 差分设计，持久化仅存承诺值和盲因子，份额明文在邮件分发后丢弃，服务器不保存可恢复密钥的敏感数据。资产经 AES-256 加密存储，加密密钥即为被门限拆分的秘密，形成加密层与门限层双层防护。

在工程层面，前端提供四步向导、控制台面板、监护人门户和 AI 助手，降低门限密码学参数配置门槛。后端实现 PBKDF2 认证、邮件通知、继承管理和资产恢复服务链路。智能合约定义三态状态机和三种触发模式。测试覆盖 43 个核心用例，验证了伪造份额拒绝、未达门限恢复失败和时间锁触发拦截等逆向安全场景。本作品将数字遗产继承从理论转化为可运行的全栈原型，兼顾密码学安全强度与用户操作便利性，可服务于加密资产持有人、数字创作者、跨地域家庭和企业密钥托管等场景。

关键词：数字遗产；门限密码学；Shamir 秘密共享；Pedersen 承诺；同态刷新；智能合约；抗胁迫

目录

摘要	1
第一章 作品概述	3
1.1 研究背景	3
1.2 需求分析	4
1.3 项目目标	5
1.4 相关工作	5
1.5 应用前景	6
第二章 作品设计与实现	8
2.1 总体架构	8
2.2 功能模块划分	9
2.3 核心数据结构	10
2.3.1 后端 LegacyPlan 结构	10
2.3.2 Share 与 StoredShare 的差分设计	12
2.3.3 InheritanceRequest 结构	13
2.4 创建遗产计划完整流程	13
2.4.1 前端多步骤向导	13
2.4.2 后端创建流程	13
2.4.3 创建计划关键代码	14
2.5 Shamir 秘密共享实现	16
2.5.1 数学原理	16
2.5.2 代码实现	16
2.5.3 随机标量生成	18
2.6 Pedersen 同态承诺与双多项式结构	19
2.6.1 Pedersen 同态承诺	19
2.6.2 双多项式份额结构	20
2.6.3 同态份额刷新（零和多项式）	20

2.7	可验证秘密共享与零知识证明	21
2.7.1	第一层：份额承诺验证	21
2.7.2	第二层：Pedersen 同态承诺验证	22
2.7.3	Schnorr 零知识证明（未来扩展方向）	23
2.7.4	综合验证流程	24
2.8	VSS 与未来扩展：Schnorr 协议交互时序	25
2.9	资产加密与恢复	25
2.9.1	加密层设计	26
2.9.2	门限层设计	26
2.9.3	恢复链路	27
2.10	触发模式设计	28
2.10.1	后端触发模式	28
2.10.2	智能合约触发模式	28
2.11	后端 API 设计	29
2.11.1	完整接口列表	29
2.11.2	Express 应用初始化	30
2.12	智能合约设计	31
2.12.1	数据结构	31
2.12.2	状态机设计	31
2.12.3	核心函数实现	31
2.12.4	合约安全分析	33
2.12.5	合约扩展方向	34
2.13	密码学模块的安全边界分析	34
2.13.1	随机数生成的安全性	34
2.13.2	私钥材料的生命周期	34
2.13.3	椭圆曲线运算的正确性	35
2.13.4	代码质量与安全审计要点	35
2.13.5	与传统 SSS 实现的兼容性	35
2.14	前端交互设计与组件架构	36
2.14.1	前端技术栈与项目结构	36
2.14.2	页面导航结构	36
2.14.3	前端 API 服务层	36
2.14.4	组件架构	36
2.14.5	前端 API 服务层	40
2.14.6	前端状态管理策略	40

2.14.7	前端安全最佳实践	41
2.15	后端服务的设计模式	41
2.15.1	仓储模式 (Repository Pattern)	41
2.15.2	工厂方法 (Factory Method)	41
2.15.3	观察者模式 (Observer Pattern)	41
2.15.4	验证器链 (Validator Chain)	42
2.15.5	模板方法 (Template Method)	42
2.15.6	策略模式 (Strategy Pattern) 的潜在应用	42
2.16	AI 助手实现	42
2.16.1	意图识别引擎	42
2.16.2	对话状态管理	43
2.16.3	命令执行管道	43
2.17	邮件通知系统	43
2.17.1	邮件发送架构	44
2.17.2	邮件通知触发点	44
2.18	有限域模运算实现	45
2.18.1	模运算基础函数	45
2.18.2	域参数选择	46
2.19	用户认证与密钥派生	46
2.20	开发环境与工程管理	47
2.20.1	Monorepo 项目组织	47
2.20.2	技术依赖与版本管理	47
2.20.3	开发 workflow	47
2.21	部署架构与生产化路径	47
2.21.1	容器化部署方案	47
2.21.2	安全加固建议	47
2.21.3	监控与运维	49
2.22	系统完整生命周期与数据流	49
第三章	作品测试与分析	50
3.1	测试环境与工具链	50
3.1.1	开发与测试环境	50
3.2	测试策略与层次	50
3.3	核心测试用例	50
3.4	合约测试实现	53

3.5	安全性分析	55
3.5.1	威胁模型	55
3.5.2	威胁防护矩阵	55
3.5.3	安全属性的形式化表述	56
3.5.4	关键安全属性验证	56
3.5.5	攻击路径分析	57
3.5.6	密码学安全强度分析	57
3.6	密码学功能正确性验证	58
3.6.1	Shamir 秘密共享正确性	58
3.6.2	同态刷新正确性验证	58
3.6.3	主承诺同态验证精度测试	58
3.6.4	密码学操作性能基准	59
3.6.5	规模扩展性测试	59
3.6.6	API 响应时间测试	59
3.6.7	邮件模板生成性能	60
3.7	性能分析	60
3.7.1	计算开销分布	60
3.7.2	存储分析	62
3.7.3	并发安全分析	62
3.7.4	网络通信安全	63
3.7.5	隐私保护增强方向	63
3.7.6	区块链层级的安全考量	63
第四章	创新性说明	64
4.1	场景创新：数字遗产全生命周期模型	64
4.2	技术创新：双层可验证秘密共享机制	64
4.3	工程创新：全栈安全工程实践	65
4.4	可扩展性创新	65
4.5	与现有方案的详细对比	66
4.6	工程实现的创新价值	67
4.7	社会价值与应用前景展望	67
第五章	总结	68
5.1	工作总结	68
5.2	不足与展望	68
5.3	竞赛演示方案	69

5.3.1	演示场景一：正常创建与继承流程	70
5.3.2	演示场景二：伪造份额拒绝	70
5.3.3	演示场景三：时间锁提前触发拒绝	70
5.3.4	演示场景四：未达门限恢复拒绝	70
5.3.5	演示场景五：AI 助手引导操作	70
5.3.6	演示技术准备	71
5.3.7	演示脚本示例	71
参考文献		72

第一章 作品概述

1.1 研究背景

随着数字经济和区块链技术的快速发展，越来越多的重要资产以数字形式存在。根据 Chainalysis 2024 年报告，全球加密货币持有者已超过 4.2 亿人，仅比特币的市值就曾突破 1 万亿美元。与此同时，云端存储账户、社交媒体账号、数字版权作品、NFT 以及智能合约控制权限等数字资产形态不断涌现。与传统实物资产不同，这些数字资产通常依赖密码、私钥或访问令牌进行控制，一旦持有人发生意外、长期失联或失去操作能力，相关资产可能永久无法访问。

传统遗产继承主要依赖法律文书、公证机构和平台申诉机制完成资产交接。然而这些方式在数字资产场景中面临多重困境：首先，私钥和助记词的本质是不可伪造的随机字符串，无法像实物财产那样通过法院判决强制转移；其次，如果持有人提前向第三方公开密码或助记词，会带来持续性的泄露风险；再次，中心化平台的“账户恢复”流程往往需要身份证明、死亡证明等材料，处理周期长且用户体验差；最后，如果完全由持有人独自掌握所有凭据，持有人本身就成为整个资产体系的单点失效点。

对于这一问题，学术界和工业界已经展开了相关研究。Shamir 于 1979 年提出的秘密共享方案为门限恢复提供了理论基础，Feldman 于 1987 年提出的可验证秘密共享增强了方案在敌手模型下的安全性，Schnorr 于 1991 年提出的签名与身份验证方案为份额的零知识验证提供了高效工具。在工业实践方面，以太坊的社交恢复钱包、Chainlink 的去中心化预言机、以及各类密钥管理服务（KMS）也从不同角度探索了密钥托管与恢复的可行方案。

然而，现有方案往往侧重于单一维度的安全性，未将资产托管、条件触发、份额验证、密钥恢复和资产释放等环节整合为统一的生命周期模型。数字遗产继承不仅是一个密码学问题，更是一个涉及安全工程、用户体验和社会规则的复杂系统工程。

针对上述挑战，本作品设计并实现了一套面向数字遗产场景的安全继承系统。系统利用门限密码学将主密钥拆分为多个份额，由不同监护人分别保管；只有达到预设门限后才能恢复资产访问能力。同时结合可验证秘密共享、零知识证明以及智能合约状态约束机制，实现数字资产从托管、触发到恢复的全生命周期安全管理。

1.2 需求分析

数字遗产继承系统需要同时满足安全性、可用性和可审计性要求。经过对场景的深入分析，本作品总结出以下核心需求：

1. 继承可达性需求

当资产持有人失能、失联或离世后，合法继承人能够在满足预设条件后恢复资产访问权限。系统必须具备明确的触发条件和可靠的恢复流程，使得继承过程不因持有人无法参与而中断。同时，持有人本人在正常状态下仍能自由管理和更新遗产计划。

2. 隐私保护需求

在继承条件触发之前，任何单一监护人、服务器或第三方机构都不能获取完整资产信息。具体而言：第一，服务器存储的计划数据中不应包含份额明文值；第二，单个监护人持有的份额不足以恢复主密钥；第三，加密资产包的密钥本身也被门限拆分，攻击者必须同时获得密文和足够份额才能解密资产。

3. 抗单点失效需求

系统不能依赖单个用户、单个设备或单个服务节点保存核心秘密。即使部分监护人失联、拒绝配合或恶意操作，只要满足门限数量的诚实监护人参与，系统仍能完成恢复。同时，系统的后端服务本身也不应成为新的单点失效源——即使服务器数据库被读取，攻击者也无法获得主密钥或份额明文。

4. 份额真实性需求

提交的恢复份额必须能够被验证，防止恶意用户提交伪造数据干扰恢复过程。验证机制应具有密码学强度，而非仅依赖格式校验。具体来说，系统必须能够验证：份额值是否与创建时的承诺匹配；份额是否确实来源于原始秘密共享多项式；提交者是否确实知道份额值（而非截获了承诺数据）。

5. 审计追踪需求

系统应能够记录计划创建、继承申请、份额提交和资产恢复等关键操作，为后续审计提供依据。审计日志应包含操作时间、操作者身份、操作类型和操作结果，但不记录敏感数据（如份额明文值或主密钥）。

6. 可用性需求

系统的用户界面应降低门限密码学参数配置的专业门槛。资产持有人无需理解 Shamir 多项式的数学原理即可创建遗产计划；监护人只需简单的复制粘贴操作即可提交份额；继承人可以通过清晰的操作指引完成继承流程。

7. 兼容性需求

系统应支持多种数字资产类型，包括但不限于加密货币私钥、云存储账号、加密文件和智能合约权限，并提供统一的资产元数据描述格式。

1.3 项目目标

项目目标可以从功能目标、安全目标和工程目标三个维度进行分解。下表给出了详细的目标说明：

表 1.1: 项目目标分解

目标类型	具体说明
功能目标	支持用户注册与登录（含验证码机制）；支持创建遗产计划并配置资产、监护人、门限和触发条件；支持发起继承请求并进入份额收集状态；支持监护人提交份额并完成密码学验证；支持达到门限后自动恢复主密钥并解密资产；支持 AI 助手通过自然语言引导用户完成操作。
安全目标	主密钥不在任何持久化存储中以明文形式存在；份额明文仅在创建时发送给监护人，不在计划存储中持久化；提交份额前必须通过承诺验证和多项式一致性验证；少于门限数量的份额无法恢复主密钥；加密资产包使用 AES 加密，密钥即是被门限拆分的秘密。
工程目标	前后端分离架构，接口清晰，代码分层明确；密码学模块独立封装，可单独测试和迁移；智能合约表达链上状态约束，事件驱动状态转换；项目文档和报告完整，满足竞赛评审要求。
展示目标	支持从创建计划到继承完成的端到端演示，并能展示篡改份额失败、未达门限恢复失败、时间锁未到期触发失败等多种安全场景。演示过程能够清晰体现密码学验证机制的实际效果。

1.4 相关工作

现有数字资产继承方案大致可以归纳为以下几类，每类方案有其适用场景和局限性：

1. 传统遗嘱与公证

将密码或助记词作为财产线索写入法律文书，由律师或公证机构保管并在持有人去世后交付继承人。该方法的问题在于：密码材料可能在保管期间泄露；公证机构本身可能成为攻击目标；法律程序周期长且成本高；跨国数字资产面临不同司法辖区的法律冲突。

2. 中心化托管与平台恢复

由第三方机构（如交易所、云服务商）保存备份或在认证后协助恢复账户。例如

Coinbase 的“遗产托管”功能和 Google 的“非活跃账户管理器”。这类方案将信任集中在单一机构，存在平台运营风险、政策变更风险和内部人员作恶风险。

3. 硬件钱包恢复服务

将密钥片段交予设备厂商或合作机构保管，如 Ledger Recover。该方法依赖硬件厂商的信誉和安全基础设施，适用范围局限于特定硬件设备支持的币种和网络。

4. 链上多签与社交恢复钱包

利用区块链智能合约实现多签名控制，由多个签名方共同管理资产。代表方案包括 Argent 社交恢复钱包和 Gnosis Safe 多签钱包。这类方案安全性高，但通常仅覆盖链上资产，难以处理云存储账号、加密文件等广义数字资产。

上述方案各有价值，但在隐私、成本、通用性和抗单点失败方面仍存在不足。传统遗嘱可能提前暴露敏感信息，中心化服务引入平台风险，硬件恢复服务适用范围较窄，链上多签又难以覆盖云端账号和加密文件。本作品结合门限密码学和可验证秘密共享，使其既能处理链上资产，也能处理更广义的数字凭据，同时在后端不持久化份额明文，降低服务器泄露造成的二次伤害。

1.5 应用前景

数字遗产管家具备广泛的应用前景，可服务于以下场景：

- 加密货币长期持有人：将比特币、以太坊等钱包私钥、助记词或冷钱包访问说明通过系统加密托管，设置多位监护人和恢复门限，避免因个人失联导致数字资产永久丢失。据估算，由于私钥丢失而永远无法访问的比特币已超过 370 万枚，总价值超过 1000 亿美元。
- 数字内容创作者：摄影师、作家、音乐人、研究者可将未公开作品、加密原稿、云盘账号和版权凭据设置继承规则，确保知识产权在本人无法管理时能够按照预设意愿进行转移。
- 企业密钥管理：企业可将关键运维密钥（如服务器 SSH 密钥、数据库主密钥、CI/CD 凭据）分片给 CTO、安全负责人和运维负责人，设置 2-of-3 恢复门限，降低单人离职或失联导致的业务中断风险。
- 跨地域家庭：居住在不同国家的家庭成员可分别担任监护人，使资产释放不依赖单一司法辖区或单一平台的政策变动。
- 去中心化自治组织（DAO）：开源社区和去中心化组织可管理项目多签密钥、合约升级权限、社区金库和长期归档凭据，确保组织治理不因核心成员流失而瘫痪。
- 养老与长期归档：个人可将数字资产设置为时间锁模式，在设定的延迟期（如 365 天）后自动进入可继承状态，适用于养老规划和档案长期保存。

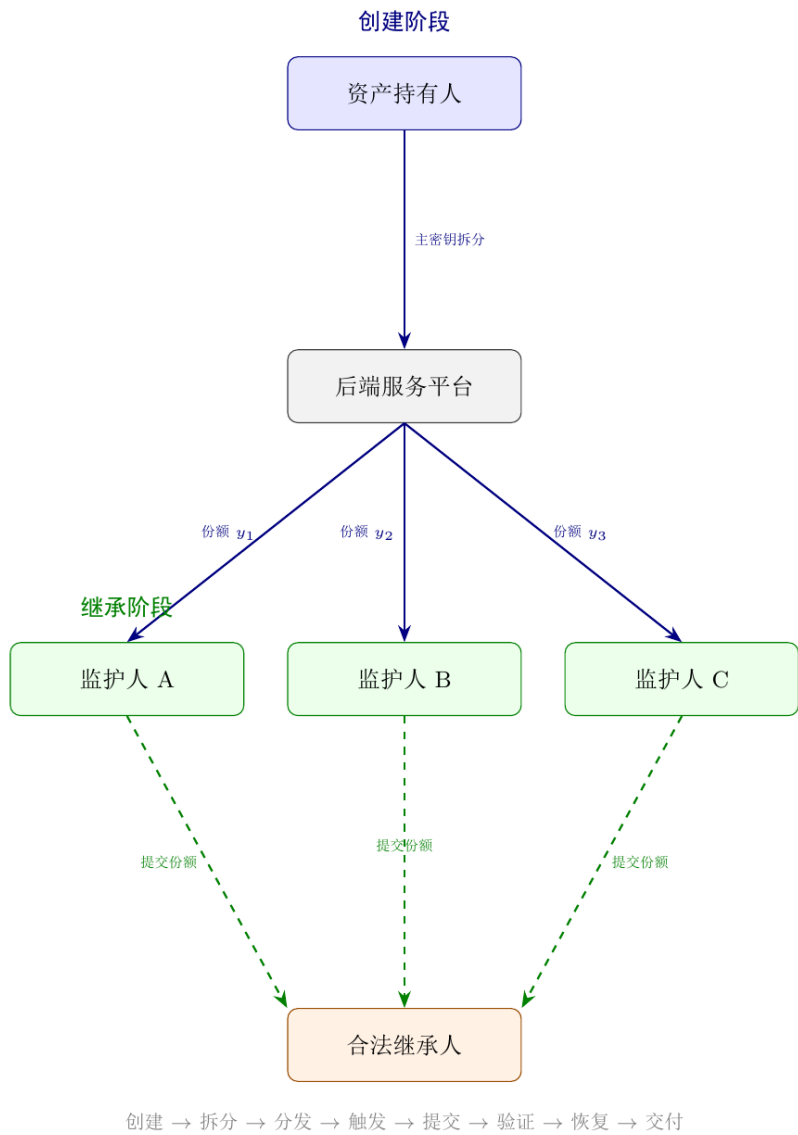


图 1.1: 数字遗产管家系统全景: 密钥拆分为 n 份额分发给监护人, 资产加密后交后端存储; 继承时达到 t 份额即可恢复

第二章 作品设计与实现

2.1 总体架构

系统按照用户层、API 层、密码学与业务逻辑层、存储层、智能合约层进行分层设计。各层职责明确，层间通过明确的接口进行通信。

用户层 (Presentation Layer): 基于 React 18 + TypeScript + TailwindCSS 构建的单页面应用 (SPA)，使用 Vite 作为构建工具。该层负责创建计划向导、控制台面板、继承流程页面、监护人门户和 AI 助手对话框等可视化界面。采用 Framer Motion 实现流畅的页面过渡动画，提升用户体验。

API 层 (API Layer): 基于 Node.js + Express + TypeScript 构建的 RESTful API 服务，监听 3000 端口。该层负责 HTTP 请求解析、参数校验、路由分发、认证授权和响应格式化。使用 CORS 中间件支持跨域请求，JSON body 大小限制为 15MB 以支持文件上传场景。

核心服务层 (Core Service Layer): 包含三个主要子模块——密码学模块 (crypto/shamir.ts) 负责 Shamir 秘密共享、Pedersen 同态承诺生成与验证、Pedersen 同态承诺生成与验证、主密钥生成、AES 加密解密；业务服务模块 (services/legacyPlanService.ts) 负责遗产计划生命周期管理、份额分发、继承请求处理、资产恢复逻辑和邮件通知；用户服务模块 (services/userService.ts) 负责用户注册、PBKDF2 密码哈希、登录验证码和用户搜索。

存储层 (Storage Layer): 使用本地 JSON 文件进行持久化，存储 plans.json、requests.json、users.json 和 verificationCodes.json 四个数据文件。每个文件均由相应的服务模块管理读写操作，确保数据一致性。存储层设计中最重要安全原则是：计划文件中的 shares 字段使用 StoredShare 类型，仅保存承诺值和证明材料，不保存份额明文 value 值。

智能合约层 (Smart Contract Layer): 基于 Solidity 0.8.x 编写的 LegacyContract 合约，定义 PlanStatus (Active、Triggered、Completed) 状态机和 TriggerMode (TimeLock、Consensus、Hybrid) 触发模式。合约在链上记录计划元信息、监护人地址与承诺值映射、份额提交状态和状态转换事件，为未来链上部署提供接口。

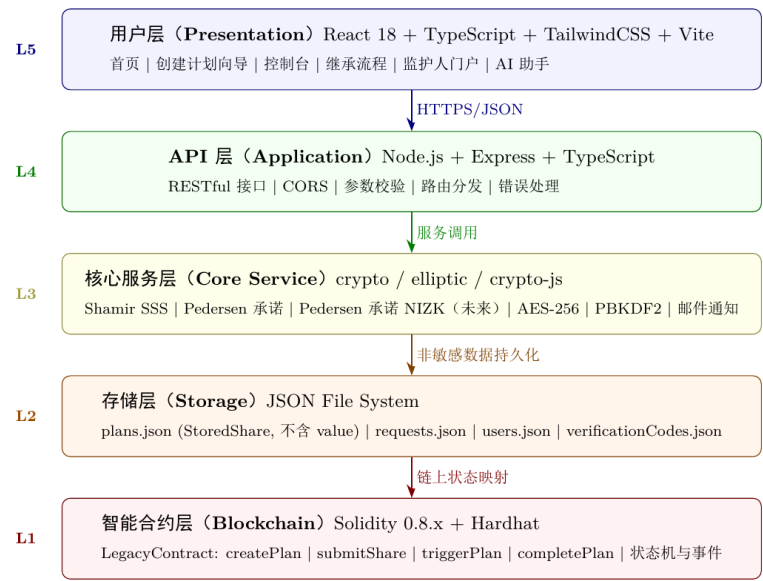


图 2.1: 系统分层架构图：自上而下为用户层、API 层、核心服务层、存储层和智能合约层，层间通过明确接口通信

2.2 功能模块划分

系统各功能模块及其关键文件和职责如下表所示。模块按照前端、后端和合约三个维度进行划分，每个模块均有明确的功能边界和接口契约。

表 2.1: 功能模块详细说明

模块	关键文件	功能说明
应用入口	App.tsx, main.tsx	配置 React Router 路由表，组织首页、创建计划、控制台、继承、监护人、AI 助手和登录页面导航。
计划创建	pages/ CreatePlan.tsx	四步向导式表单（资产添加、监护人配置、触发条件、确认创建），支持搜索已注册用户和文件上传。
控制台	pages/ Dashboard.tsx	以卡片形式展示用户相关的所有遗产计划（创建、继承、监护），支持详情查看和份额刷新触发。
继承流程	pages/ Inheritance.tsx	输入计划 ID 发起继承，查看份额收集进度和各监护人提交状态，达到门限后展示恢复资产。

模块	关键文件	功能说明
监护人门户	pages/ Guardian.tsx	监护人输入计划 ID、监护人 ID 和份额值提交至后端验证，验证通过后计入计数，界面极简。
AI 助手	components/ AIAssistant. tsx	内嵌自然语言对话界面，支持在聊天中完成创建计划、添加资产、配置监护人和发起继承等操作。
API 入口	index.ts	Express 应用配置（CORS、JSON 解析、路由注册）和全局错误处理，监听 3000 端口。
密码学模块	crypto/shamir. ts	实现 Shamir SSS 拆分/恢复、Pedersen 承诺生成/验证、AES 加解密等核心密码学原语。
业务服务	services/ legacyPlanService. ts	管理计划 CRUD、份额邮件发送、继承请求处理、份额验证和资产恢复，采用内存 Map+JSON 持久化。
用户服务	services/ userService.ts	管理用户注册（PBKDF2 哈希）、登录验证、用户搜索和数据持久化，用户 ID 为 4 位字母数字。
邮件服务	services/ emailService. ts	封装 Nodemailer，实现份额分发、继承通知、验证码和确认邮件发送。
AI 服务	services/ aiService.ts	定义 14 种意图类型，通过正则匹配自然语言输入，维护对话上下文状态机并执行业务操作。
智能合约	LegacyContract. sol	定义 LegacyPlan 结构体和 PlanCreated 等事件，实现 createPlan、submitShare、triggerPlan、completePlan 等函数。
合约测试	LegacyContract. test.js	基于 Hardhat+Chai，覆盖计划创建、份额提交、计划触发等场景，使用 EVM 时间控制。

2.3 核心数据结构

本节给出系统中三个核心数据结构的设计意图和安全考量。

2.3.1 后端 LegacyPlan 结构

后端 LegacyPlan 结构是整个系统的核心数据模型。其最关键的的安全设计是 shares 字段使用 StoredShare 类型而非完整的 Share 类型——StoredShare 仅保存份

数字遗产管家 — 完整协议流程

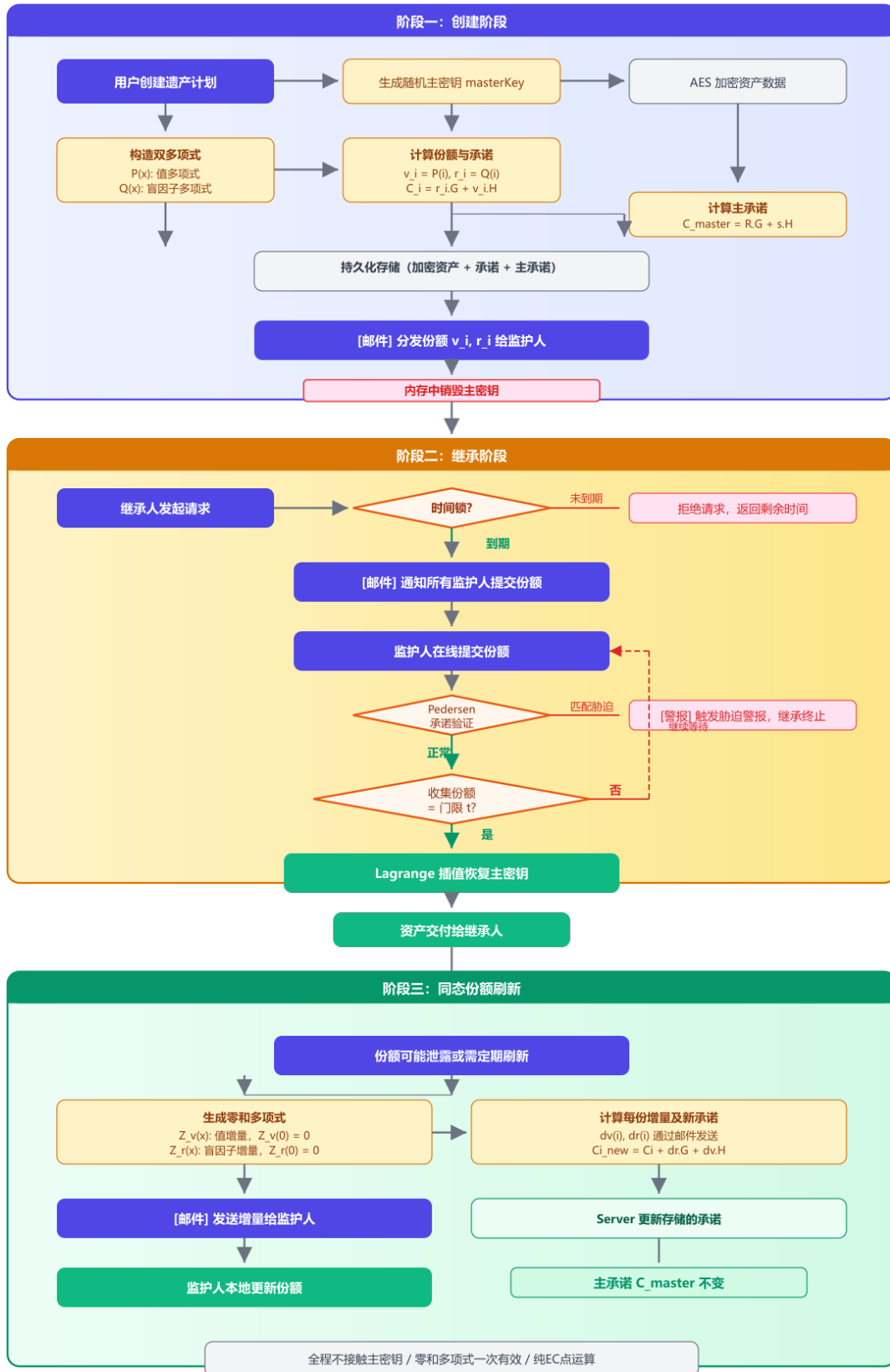


图 2.2: 系统完整协议流程——创建阶段、继承阶段与同态份额刷新阶段

额标识、索引、椭圆曲线承诺、承诺证明，不保存份额明文值 value。这意味着即使攻击者获得服务器数据库的完全读取权限，也无法直接获得用于恢复主密钥的份额值。

Listing 2.1: 后端计划核心结构定义 (legacyPlanService.ts:45-61)

```

1 export interface LegacyPlan {
2   id: string
3   name: string
4   assets: any[]           // 资产摘要信息
5   guardians: any[]       // 监护人信息（含邮箱）
6   threshold: number      // 恢复门限t
7   totalShares: number    // 总份额数n
8   triggerMode: 'consensus' | 'timed' // 触发模式
9   timeLock: number       // 时间锁天数
10  shares: StoredShare[]   // 不含value的安全份额结构
11  encryptedAssets: string // AES加密的资产包
12  masterCommitment: string //  $C_{master} = R * G + s * H$ （同态验证）
13  createdAt: string
14  updatedAt: string
15  status: 'active' | 'collecting' | 'completed'
16  creatorId?: string

```

2.3.2 Share 与 StoredShare 的差分设计

系统在密码学模块中定义了两种份额接口：Share 包含完整的份额信息用于临时计算和监护人交付，StoredShare 剔除了 value 和 blindingFactor 字段用于持久化存储。这一设计是系统”最小化存储敏感数据”原则的体现。

Listing 2.2: Share 与 StoredShare 接口定义 (shamir.ts:6-20)

```

1 export interface Share {
2   id: string           // 份额唯一标识
3   index: number        // 多项式中的x坐标 (1,2,...,n)
4   value: string        // 份额值 $y=f(x)$  — 仅临时存在于内存
5   commitment: string   // SHA256承诺
6
7 export interface StoredShare {
8   id: string
9   index: number
10  commitment: string    // SHA256承诺
11  blindingFactor: string // Pedersen承诺盲因子
12  duressCommitment?: string // 胁迫承诺
13  duressBlindingFactor?: string // 胁迫盲因子
14  // 注意：此处不包含value字段

```

2.3.3 InheritanceRequest 结构

继承请求结构用于跟踪继承流程的状态演进。其中 submittedShares 仅在内存中暂存（不持久化到 JSON 文件），用于在达到门限后立即恢复主密钥并解密资产。

Listing 2.3: 继承请求结构定义 (legacyPlanService.ts:63-76)

```
1 export interface InheritanceRequest {
2   id: string
3   planId: string
4   initiatorId: string           // 发起继承请求的用户 ID
5   heirEmail: string            // 继承人邮箱
6   guardianSignatures: string[]
7   sharesCollected: number
8   submittedGuardians: string[] // 已提交监护人ID集合（防重）
9   submittedShares?: Share[]    // 已提交份额（不持久化到文件）
10  hasDuress: boolean           // 是否检测到胁迫提交
11  status: 'pending' | 'collecting' | 'verifying' | 'completed' | 'duress'
12  createdAt: string
13  updatedAt: string
```

2.4 创建遗产计划完整流程

创建遗产计划是系统的核心业务流程入口。本节从前端交互、后端参数校验、密码学拆分、承诺生成、加密存储和份额分发六个阶段详细描述该流程。

2.4.1 前端多步骤向导

前端 CreatePlan 页面采用四步向导设计，逐步引导用户完成资产配置、监护人设置、触发条件配置和确认创建。每步包含实时表单验证和错误提示，降低用户配置门限密码学参数的专业门槛。

第一步“添加资产”支持四种资产类型（加密货币、云账号、加密文件、智能合约），每种类型有对应的输入格式校验规则。文件类型资产支持通过 FileReader API 进行本地 Base64 编码后上传。第二步“设置监护人”集成了用户搜索功能，通过调用 /api/users/search 接口查找已注册用户，自动填充监护人 ID、姓名和邮箱。同时允许指定继承人。第三步“设置触发条件”可选社会共识模式或时间锁模式（混合模式），实时展示门限配置为 t-of-n 含义的解释文本。第四步“确认创建”展示资产清单和监护人配置摘要，并提供安全提示。

2.4.2 后端创建流程

当后端 API 收到创建计划请求后，按以下步骤处理：

1. 参数校验：验证资产列表非空、监护人列表非空、门限 t 满足 $1 \leq t \leq n$ 、总份额数不超过监护人数量。
2. 主密钥生成：调用 `shamirSecretSharing.generateMasterKey()` 生成 256 位随机主密钥 K 。该方法使用拒绝采样在 `secp256k1` 标量域 $\mathbb{F}_q$ 内生成均匀分布的随机数。
3. 秘密共享拆分：以主密钥 K 为 a_0 ，生成 $t-1$ 个随机系数 $a_1, \dots, a_{t-1}$ ，构造 $t-1$ 次多项式 $f(x) = a_0 + a_1x + \dots + a_{t-1}x^{t-1} \pmod{q}$ 。对 $i = 1$ 到 n ，计算 $y_i = f(i)$ ，得到 n 个份额点 (i, y_i) 。
4. 承诺与证明生成：为每个份额生成椭圆曲线承诺 $Y_i = y_iG$ 、多项式系数承诺 $A_j = a_jG$ 和 Schnorr 证明（未来扩展） (R_i, z_i) 。承诺用于后续验证份额完整性，证明用于验证提交者确实知道份额值。
5. 资产加密：调用 `CryptoJS.AES.encrypt()` 以主密钥 K 加密整个资产包 JSON 字符串，得到密文 C 。
6. 安全存储：将 `Share` 对象转换为 `StoredShare`（移除 `value` 字段），与资产摘要、监护人信息、加密资产包等组装为 `LegacyPlan` 结构，持久化到 `plans.json`。
7. 份额分发：通过邮件服务将包含 `value` 的完整份额信息发送给各监护人。邮件中包含计划名称、计划 ID、份额 ID、份额索引、份额值、门限和总份额数等关键信息。

2.4.3 创建计划关键代码

后端 `createPlan` 方法是整个流程的核心，以下摘录关键代码段并进行说明：

Listing 2.4: `createPlan` 核心实现 (`legacyPlanService.ts:118-210`, 节选)

```

1 createPlan(data: {...}): LegacyPlan {
2   // 第一步：输入验证
3   if (!data.assets || data.assets.length === 0) {
4     throw new Error('至少需要添加一个资产')
5   }
6   if (data.threshold < 1 || data.threshold > data.totalShares) {
7     throw new Error('门限阈值必须在 1 到总份额数之间')
8   }
9   if (data.totalShares > data.guardians.length) {
10    throw new Error('总份额数不能超过监护人数量')
11  }
12  // 第二步：生成主密钥并进行秘密共享
13  const masterKey = shamirSecretSharing.generateMasterKey()
14  const shares = shamirSecretSharing.split(masterKey, data.totalShares, data.threshold)
15
16  // 第三步：加密资产（密钥为主密钥）
17  const encryptedAssets = shamirSecretSharing.encryptAsset(data.assets, masterKey)
18
19  // 第四步：安全存储 — Share转StoredShare（移除value字段）
20  const storedShares: StoredShare[] = shares.map(share => ({
    id: share.id, index: share.index,
    commitment: share.commitment,
    proof: share.proof,
  })

```

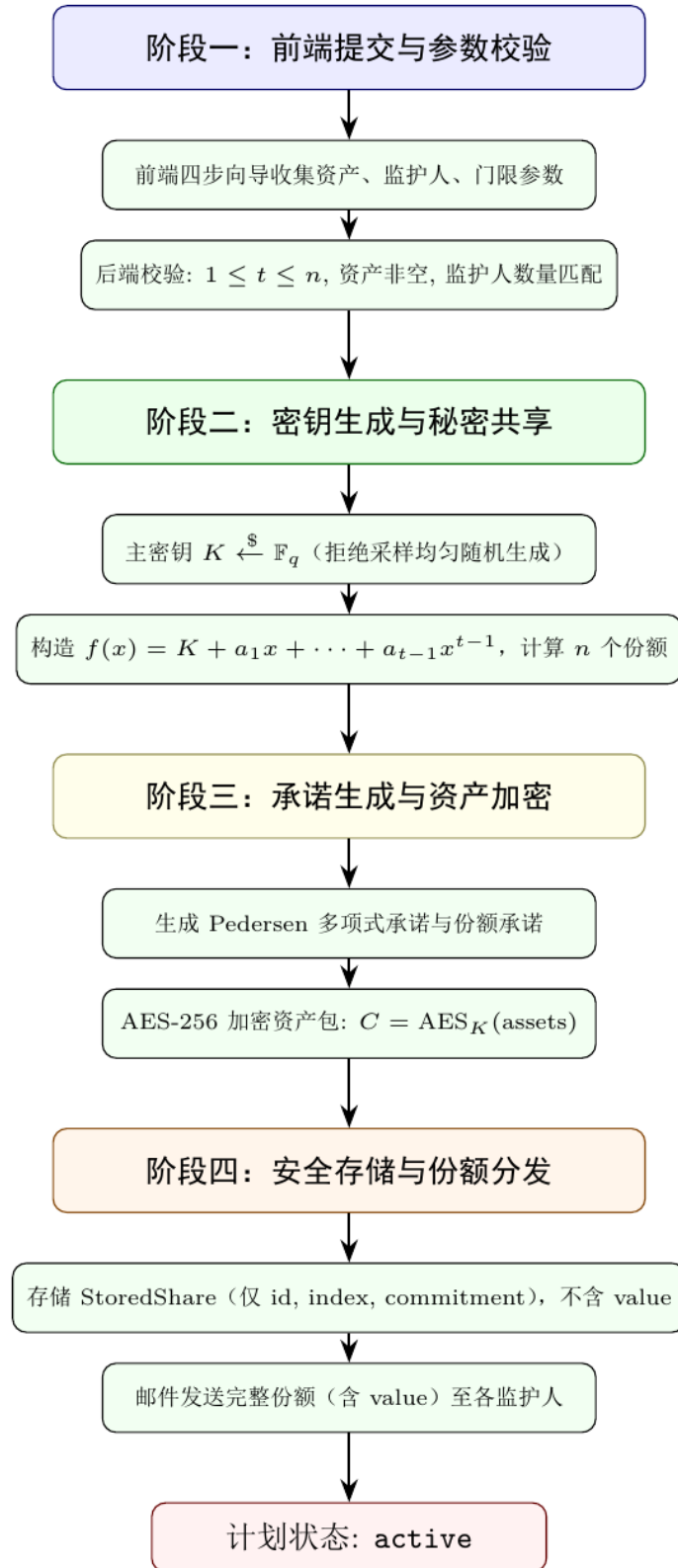


图 2.3: 遗产计划创建流程：四个阶段自上而下顺序执行，最终得到不含份额明文的持久化计划和已分发给监护人的完整份额

```

21     polynomialCommitments: share.polynomialCommitments
22     // 关键: 此处不包含 share.value
23   })
24
25   // 第五步: 组装并持久化计划
26   const plan: LegacyPlan = { id: uuidv4(), ..., shares: storedShares, encryptedAssets, status
    : 'active' }
27   this.plans.set(plan.id, plan)
28   this.saveData()
29
30   // 第六步: 邮件分发完整份额 (含value) 给监护人
31   this.sendShareEmails(plan, shares)
32   return plan

```

2.5 Shamir 秘密共享实现

Shamir Secret Sharing 是本作品的密码学基础。本节详细描述秘密共享的原理、实现和在系统中的应用。

2.5.1 数学原理

Shamir 秘密共享的核心思想是在有限域 $\mathbb{F}_q$ 上构造 $t - 1$ 次随机多项式:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{q}$$

其中秘密 $S = a_0 = f(0)$ 。系统随机生成 $t - 1$ 个系数 $a_1, \dots, a_{t-1}$, 为 n 个参与者分别计算份额点 $(x_i, y_i = f(x_i))$ 。由于 $t - 1$ 次多项式由 t 个点唯一确定, 任意 t 个份额可以重构多项式并计算 $f(0)$ 恢复秘密; 而任意 $t - 1$ 个份额只提供 $t - 1$ 个点, 无法唯一确定 $t - 1$ 次多项式, 因此从信息论角度获得的信息量为零。

恢复阶段使用拉格朗日插值公式:

$$S = f(0) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j} = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{q}$$

本作品在使用域方面有一个关键设计决策: 没有混用椭圆曲线基域和群阶, 而是在 secp256k1 的标量域 $\mathbb{F}_q$ (其中 q 为群阶) 上进行 Shamir 多项式运算。这使得 Pedersen 承诺中的椭圆曲线等式 $y_i G = \sum_{j=0}^{t-1} x_i^j A_j$ 自然成立, 无需额外的域兼容性处理。

2.5.2 代码实现

Listing 2.5: Shamir 秘密共享拆分实现 (shamir.ts:61-91)

```

1 split(secret: string, n: number, t: number): Share[] {
2   // 解析秘密为域元素
3   const secretNum = this.parseScalar(secret)
4
5   // 生成t-1个随机系数: a1, a2, ..., a_{t-1}
6   const coefficients: bigint[] = [secretNum] // a0 = secret
7   for (let i = 1; i < t; i++) {
8     coefficients.push(this.generateRandomScalar())
9   // 计算多项式系数承诺 Ak = ak * G (用于Pedersen承诺验证)
10  const polynomialCommitments = coefficients.map((coeff) =>
11    this.pointToHex(this.scalarBaseMult(coeff))
12  )
13
14  // 为每个参与者计算份额 (i, f(i))
15  const shares: Share[] = []
16  for (let i = 1; i <= n; i++) {
17    const x = BigInt(i)
18    const y = this.evaluatePolynomial(coefficients, x) // f(i)
19    const share: Share = {
20      id: uuidv4(), index: i,
21      value: this.formatScalar(y),
22      commitment: '',
23      polynomialCommitments,
24    // 生成份额承诺
25    share.commitment = this.generateCommitment(share)
26    share.proof = this.generateShareProof(share, polynomialCommitments)
27    shares.push(share)
28  }
29  return shares

```

多项式求值采用 Horner 方法的变体:

Listing 2.6: 多项式求值实现 (shamir.ts:241-248)

```

1 private evaluatePolynomial(coefficients: bigint[], x: bigint): bigint {
2   let result = BigInt(0)
3   for (let i = 0; i < coefficients.length; i++) {
4     // result += a_i * x^i (mod q)
5     const term = this.modField(coefficients[i] * this.modPow(x, BigInt(i), this.fieldOrder))
6     result = this.modField(result + term)
7   }
8   return result

```

份额合并采用拉格朗日插值在有限域上的直接实现:

Listing 2.7: 拉格朗日插值恢复秘密 (shamir.ts:325-358)

```

1 function combineSharesInField(shares: Share[], fieldOrder: bigint): string {
2   if (shares.length < 1) {
3     throw new Error('Need at least 1 share to reconstruct')
4   }
5   const sortedShares = [...shares].sort((a, b) => a.index - b.index)
6   let secret = BigInt(0)

```

```

6   for (let i = 0; i < sortedShares.length; i++) {
7       const share = sortedShares[i]
8       const x = BigInt(share.index), y = BigInt('0x' + share.value)
9       let numerator = BigInt(1), denominator = BigInt(1)
10      for (let j = 0; j < sortedShares.length; j++) {
11          if (i !== j) {
12              const xj = BigInt(sortedShares[j].index)
13              numerator = mod(numerator * -xj, fieldOrder) // 分子: Prod(-xj)
14              denominator = mod(denominator * (x - xj), fieldOrder) // 分母: Prod(xi-xj)
15          }
16          const lagrangeCoeff = mod(
17              numerator * modInverse(denominator, fieldOrder),
18              fieldOrder
19          )
20          secret = mod(secret + mod(y * lagrangeCoeff, fieldOrder), fieldOrder)
21      }
22      return secret.toString(16).padStart(HEX_SCALAR_LENGTH, '0')

```

其中 modInverse 使用扩展欧几里得算法计算模逆元:

Listing 2.8: 扩展欧几里得算法计算模逆元 (shamir.ts:364-381)

```

1   function modInverse(a: bigint, m: bigint): bigint {
2       let oldR = m, r = mod(a, m)
3       let oldS = BigInt(0), s = BigInt(1)
4       while (r !== BigInt(0)) {
5           const quotient = oldR / r
6           [oldR, r] = [r, oldR - quotient * r]
7           [oldS, s] = [s, oldS - quotient * s]
8       }
9       if (oldR !== BigInt(1)) {
10          throw new Error('Value has no modular inverse')
11      }
12      return mod(oldS, m)

```

2.5.3 随机标量生成

密码学安全随机数生成是整个系统安全性的基石之一。本作品使用拒绝采样在有限域 $\mathbb{F}_q$ 内生成均匀分布的随机标量，避免了简单取模带来的微小偏差:

Listing 2.9: 拒绝采样生成均匀随机标量 (shamir.ts:250-262)

```

1   private generateRandomScalar(): bigint {
2       const range = this.fieldOrder - BigInt(1)
3       const bytesLength = Math.ceil(this.fieldOrder.toString(2).length / 8)
4       const sampleSpace = BigInt(1) << BigInt(bytesLength * 8)
5       const limit = sampleSpace - (sampleSpace % range) // 拒绝边界
6
7       while (true) {
8           const candidate = BigInt('0x' + randomBytes(bytesLength).toString('hex'))
9           if (candidate < limit) { // 在拒绝边界内才接受
10              return (candidate % range) + BigInt(1) // 排除0, 范围为[1, q-1]

```

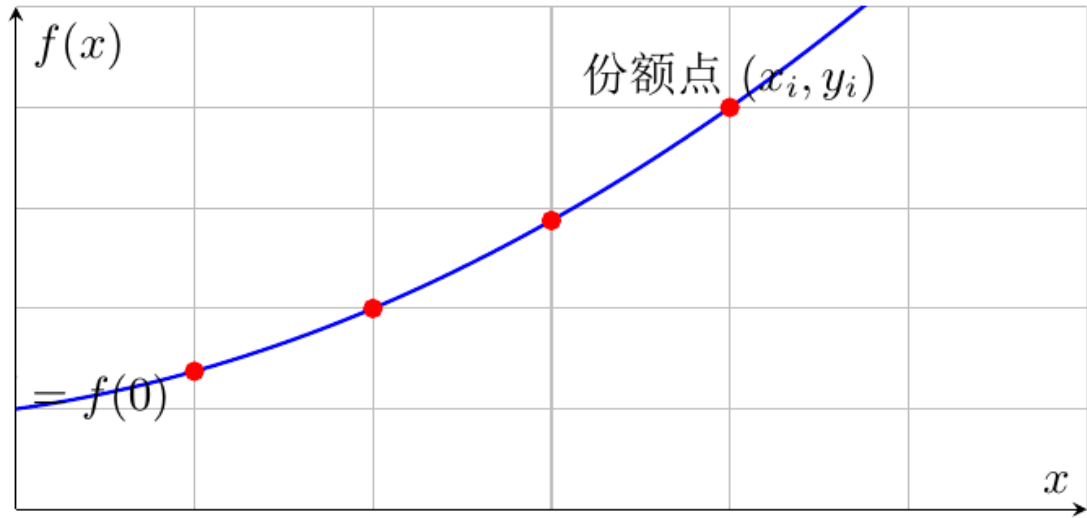


图 2.4: Shamir 秘密共享多项式示意图 ($t=3, n=4$): 任意 3 个点可唯一确定二次多项式并恢复 $S=f(0)$

2.6 Pedersen 同态承诺与双多项式结构

2.6.1 Pedersen 同态承诺

前述 Shamir 秘密共享实现了密钥的门限拆分与恢复，但 server 需要验证监护人提交的份额是否正确，又不能直接看到份额值（否则 server 即可自行恢复主密钥）。Pedersen 承诺解决了这一矛盾——在公开份额承诺的同时隐藏份额值本身。

承诺公式。给定 secp256k1 椭圆曲线上两个独立的生成点 G 和 H ($H = G \cdot \text{SHA256}(\text{seed})$ ，离散对数关系未知)，对份额值 v 和盲因子 r 的承诺为：

$$C(v, r) = r \cdot G + v \cdot H$$

安全属性：

- 完美隐藏 (Perfect Hiding)：对于任意 C 和任意 v ，存在 r 使得 $C = rG + vH$ ，即 C 不泄露 v 的任何信息。
- 计算绑定 (Computational Binding)：找到不同的 (r', v') 使得 $C(r', v') = C(r, v)$ 等价于解椭圆曲线上的离散对数问题 $\log_G H$ 。

同态加法性质：

$$C(v_1, r_1) + C(v_2, r_2) = C(v_1 + v_2, r_1 + r_2)$$

这一性质是份额刷新功能的核心密码学基础。

2.6.2 双多项式份额结构

标准 Shamir 方案仅拆分份额值，无法在不暴露值的情况下验证份额正确性。本系统引入第二个盲因子多项式 $Q(x)$ ，构成双多项式结构：

$$\begin{cases} P(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{q} \\ Q(x) = R + b_1x + b_2x^2 + \cdots + b_{t-1}x^{t-1} \pmod{q} \end{cases}$$

其中 $P(0) = s$ （主密钥）， $Q(0) = R$ （随机主盲因子）。份额 i 包含值 $v_i = P(i)$ 和盲因子 $r_i = Q(i)$ 。对应的 Pedersen 承诺为 $C_i = r_iG + v_iH$ 。主承诺 $C_{\text{master}} = RG + sH$ 公开存储。

同态可验证性——利用 Lagrange 系数 λ_j 在 EC 点上直接运算：

$$\Sigma(\lambda_j \cdot C_{i_j}) = C_{\text{master}}$$

任意第三方（包括 server）可验证一组份额是否正确对应主密钥，无需知道份额值 v_i 和盲因子 r_i 。

2.6.3 同态份额刷新（零和多项式）

创新点：在 server 完全不存储主密钥的前提下，实现安全的份额刷新轮换。由于主密钥在创建后立即销毁，传统的“解密 → 重新拆分”方法不可用。

原理。生成两个 $t-1$ 次零和多项式 $Z_v(x)$ 和 $Z_r(x)$ ，满足 $Z_v(0) = Z_r(0) = 0$ ：

- 新份额值 $v'_i = v_i + Z_v(i)$ （监护人本地计算）
- 新盲因子 $r'_i = r_i + Z_r(i)$ （监护人本地计算）
- 新承诺 $C'_i = C_i + Z_r(i) \cdot G + Z_v(i) \cdot H$ （server 公开计算，纯 EC 点运算）
- 主承诺不变 $C'_{\text{master}} = C_{\text{master}}$ （因为 $Z(0) = 0$ ）

零和多项式每次刷新随机生成，一次有效。与需存储主密钥的传统方案相比，本方法具有以下安全优势：

表 2.2: 同态刷新与传统方案的安全对比

对比项	传统方案（需存储主密钥）	本方案（同态刷新）
Server 密钥存储	需加密存储主密钥	全程不接触主密钥
刷新计算	解密 → 重拆分 → 重新加密	纯 EC 点运算
通信内容	发送完整新份额（机密）	仅发送公开增量
安全性	密钥泄露 → 全盘暴露	零和多项式一次有效

核心代码实现如下：

Listing 2.10: 零和多项式同态刷新增量生成 (shamir.ts)

```

1 generateRefreshDeltas(n: number, t: number): ShareDelta[] {
2   const zvCoeffs: bigint[] = [BigInt(0)]    // Z_v(0) = 0
3   const zrCoeffs: bigint[] = [BigInt(0)]    // Z_r(0) = 0
4   for (let i = 1; i < t; i++) {
5     zvCoeffs.push(generateRandomCoefficient())
6     zrCoeffs.push(generateRandomCoefficient())
7   for (let i = 1; i <= n; i++) {
8     const x = BigInt(i)
9     deltas.push({
10      index: i,
11      valueDelta: evaluatePoly(zvCoeffs, x).toString(16),
12      blindingDelta: evaluatePoly(zrCoeffs, x).toString(16),
13    })
14  }
15  return deltas

```

Listing 2.11: 承诺同态刷新计算 (shamir.ts)

```

1 computeRefreshedCommitment(oldCommitment: string,
2   valueDelta: string, blindingDelta: string): string {
3   const G = this.ec.g, H = this.getH()
4   const oldPoint = this.ec.keyFromPublic(oldCommitment, 'hex').getPublic()
5   const vDeltaBN = new BN(valueDelta, 16)
6   const rDeltaBN = new BN(blindingDelta, 16)
7   // C' = C + delta_r * G + delta_v * H
8   const newPoint = oldPoint.add(G.mul(rDeltaBN)).add(H.mul(vDeltaBN))
9   return newPoint.encode('hex', false)

```

2.7 可验证秘密共享与零知识证明

传统 Shamir Secret Sharing 能够实现秘密的门限拆分与恢复，但其本身并不具备份额真实性验证能力。如果某个参与者提交伪造份额、损坏份额或恶意构造错误数据，系统往往只能在秘密恢复失败后才能发现问题。这种“恢复后发现错误”的机制不仅降低系统可靠性，还可能导致继承流程中断和排查成本增加。

为了在恢复前就能验证份额真实性，本作品在 Shamir Secret Sharing 的基础上引入 Pedersen 同态承诺与 SHA256 双重份额验证机制。同时，Schnorr 零知识证明（未来扩展）作为未来扩展方向被纳入系统架构设计——其核心算法已在 shamir.ts 中预留接口，可在后续版本中启用。

2.7.1 第一层：份额承诺验证

系统为每个份额 y_i 计算椭圆曲线承诺：

$$Y_i = y_i G$$

其中 G 为 secp256k1 椭圆曲线的生成元。由于椭圆曲线离散对数问题（ECDLP）的计算困难性，攻击者无法从 Y_i 反推出 y_i ，因此系统可以安全地将 Y_i 公开存储在 `StoredShare.commitment` 字段中。

当监护人提交份额 y'_i 时，系统首先进行基础验证：重新计算 $Y'_i = y'_i G$ ，并检查 Y'_i 是否与存储的 Y_i 一致。如果不一致，说明份额值已被篡改，系统立即拒绝并返回“Invalid share value or zero-knowledge proof”错误信息。

2.7.2 第二层：Pedersen 同态承诺验证

仅验证份额与单个承诺匹配仍存在安全隐患：攻击者可能提交一个来自不同多项式（不同秘密共享实例）的合法份额值。例如，攻击者 A 可能窃取了攻击者 B 创建的某计划中的一个份额，虽然该份额的 EC 承诺与自身值一致，但它来源于另一个多项式，最终恢复出的秘密将与计划创建时的主密钥完全不同。

Pedersen 承诺通过在公开信道发布多项式系数承诺来解决这一问题：

$$A_j = a_j G \quad (j = 0, 1, \dots, t-1)$$

验证时利用椭圆曲线的同态性质：若原始多项式为 $f(x) = \sum_{j=0}^{t-1} a_j x^j$ ，则：

$$f(x_i)G = \sum_{j=0}^{t-1} a_j x_i^j G = \sum_{j=0}^{t-1} x_i^j (a_j G) = \sum_{j=0}^{t-1} x_i^j A_j$$

因此系统验证：

$$Y_i \stackrel{?}{=} \sum_{j=0}^{t-1} x_i^j A_j$$

如果等式成立，说明份额 y_i 确实来源于 $f(x)$ ；如果不成立，说明份额是伪造的或来自其他多项式。此验证保证了所有参与恢复的份额都属于同一个秘密共享实例——即当前计划创建时的原始多项式。

Listing 2.12: Pedersen 同态承诺验证实现 (shamir.ts:170-188)

```

1 private verifyPolynomialCommitment(share: Share, storedShare: StoredShare): boolean {
2   try {
3     const x = BigInt(share.index)
4     // 计算 Sum(x^i * A_i)
5     const expectedPoint = storedShare.polynomialCommitments!.reduce(
6       (point, commitment, degree) => {
7         const multiplier = this.modPow(x, BigInt(degree), this.fieldOrder)
8         const term = this.pointFromHex(commitment).mul(this.bigIntToBN(multiplier))
9         return point ? point.add(term) : term
10      }, null
11    )

```

```

12     if (!expectedPoint) return false
13     // 验证: yG == Sum(x^i * A_i)
14     const committedPoint = this.pointFromHex(storedShare.commitment)
15     return expectedPoint.eq(committedPoint)
16   } catch { return false }

```

2.7.3 Schnorr 零知识证明（未来扩展方向）

上述两层验证能保证份额的完整性和一致性，但存在一个潜在攻击面：恶意监护人可能截获网络传输中的承诺值 Y_i ，然后将其作为“自己的份额”提交。Schnorr 零知识证明（未来扩展）通过 Fiat-Shamir 变换，使得提交者能在无需与验证者交互的情况下，证明自己确实知道离散对数 y_i 而无需暴露 y_i 本身。当前系统已预留接口，以下为设计方案。

证明生成（**Prover** 端，在创建计划时由后端执行）：

1. 生成随机数 $r \in \mathbb{F}_q$ （称为 nonce）
2. 计算 nonce 承诺： $R = rG$
3. 构建证明转录本并哈希： $e = H(\text{domain_separator} \| \text{shareId} \| \text{shareIndex} \| Y_i \| A_0 \| \dots \| A_{t-1} \| R)$
4. 计算响应： $z = r + e \cdot y_i \pmod{q}$
5. 输出证明： $\pi = (R, e, z)$

证明验证（**Verifier** 端，在监护人提交份额时由后端执行）：

1. 重新计算挑战： $e' = H(\text{domain_separator} \| \text{shareId} \| \text{shareIndex} \| Y_i \| A_0 \| \dots \| A_{t-1} \| R)$
2. 检查 $e' = e$ （防止证明被重用给不同的上下文）
3. 检查椭圆曲线等式： $zG \stackrel{?}{=} R + e \cdot Y_i$

如果等式成立，则验证通过。正确性证明：

$$zG = (r + e \cdot y_i)G = rG + e \cdot (y_i G) = R + e \cdot Y_i$$

Listing 2.13: Pedersen 承诺验证实现 (shamir.ts:190-220)

```

1 private verifySchnorrProof(
2   %% Schnorr proof is planned future work - verification currently uses Pedersen commitment
3   shareId: string, shareIndex: number,
4   shareCommitment: string,
5   polynomialCommitments: string[],
6   proof: ShareProof
7 ): boolean {
8   try {
9     // 第一步：重新计算挑战值
10    const expectedChallenge = this.challengeForProof(
11      shareId, shareIndex, shareCommitment,
12      polynomialCommitments, proof.nonceCommitment
13    )

```

```

14 // 第二步：验证挑战一致性
15 if (expectedChallenge !== proof.challenge) return false
16 // 第三步：验证椭圆曲线等式  $zG = R + eY$ 
17 const responsePoint = this.scalarBaseMult(this.parseScalar(proof.response))
18 const noncePoint = this.pointFromHex(proof.nonceCommitment)
19 const sharePoint = this.pointFromHex(shareCommitment)
20 const challenge = BigInt('0x' + proof.challenge)
21 const expectedPoint = noncePoint.add(
22   sharePoint.mul(this.bigIntToBN(challenge))
23 )
24 return responsePoint.eq(expectedPoint)
25 } catch { return false }

```

挑战值的计算采用 Fiat-Shamir 变换，将交互式证明压缩为非交互式。转录本包含了所有公开参数，确保证明绑定到特定份额、特定计划和特定上下文：

Listing 2.14: Fiat-Shamir 挑战生成 (shamir.ts:222-239)

```

1 private challengeForProof(
2   shareId: string, shareIndex: number,
3   shareCommitment: string,
4   polynomialCommitments: string[],
5   nonceCommitment: string
6 ): string {
7   const transcript = [
8     'digital-legacy-sss-zkp-v1', // 域分隔符（防跨上下文重用）
9     shareId,
10    shareIndex.toString(),
11    shareCommitment,
12    ...polynomialCommitments,
13    nonceCommitment,
14  ].join('|')
15   const challenge = BigInt('0x' + CryptoJS.SHA256(transcript).toString()) % this.groupOrder
16   return this.formatScalar(challenge)

```

2.7.4 综合验证流程

系统将三层验证组织为有序的验证管道。只有三层全部通过，份额才被接受并计入收集计数。验证管道的实现如下：

Listing 2.15: 综合份额验证 (shamir.ts:109-128)

```

1 verifyShareProof(share: Share, storedShare: StoredShare): boolean {
2   // 若没有扩展证明材料，回退到基础承诺验证
3   if (!storedShare.proof || !storedShare.polynomialCommitments?.length) {
4     return this.verifyCommitment(share, storedShare.commitment)
5   }
6   // 第一层：份额承诺验证
7   if (!this.verifyCommitment(share, storedShare.commitment)) return false
8   // 第二层：Pedersen同态承诺验证

```

```

8  if (!this.verifyPolynomialCommitment(share, storedShare)) return false
9  // 第三层: Schnorr零知识证明 (未来扩展) 验证
10 return this.verifySchnorrProof(
11  %% Schnorr proof is planned future work - verification currently uses Pedersen commitment
12    storedShare.id, storedShare.index,
13    storedShare.commitment,
14    storedShare.polynomialCommitments,
15    storedShare.proof
16  )

```

因此，系统的安全验证链路为：

份额值格式校验 → 份额承诺一致性 → 多项式来源一致性 → Schnorr 知识证明（未来） → 份

任何一环失败都会导致验证终止，这使得攻击者必须同时突破三个独立的密码学障碍才能成功伪造份额。

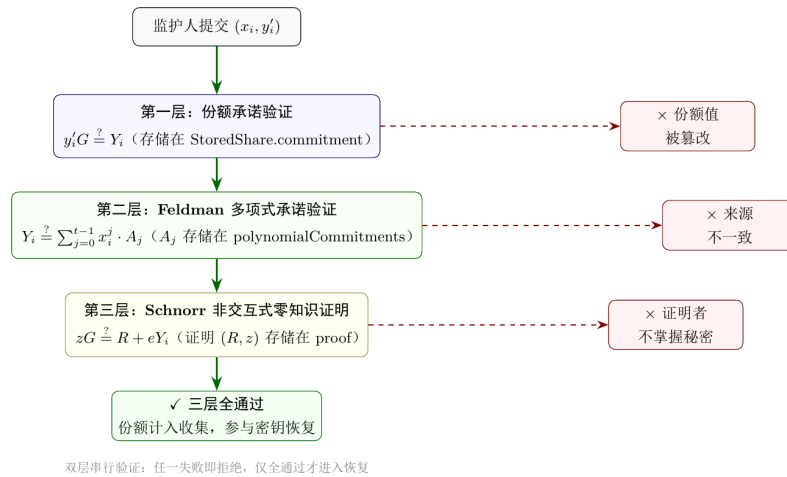


图 2.5：三层份额验证管道：自上而下串行执行，每层右侧标注了对应的失败原因，全通过后份额被接受

2.8 VSS 与未来扩展：Schnorr 协议交互时序

图2.6以时序图形式展示计划创建阶段（Prover 端）的证明材料生成和继承阶段（Verifier 端）的份额验证之间的密码学交互关系。

2.9 资产加密与恢复

为避免数字资产直接暴露在数据库或服务器中，本作品采用“对称加密 + 门限恢复”的双层安全架构。资产信息（可能包含私钥、助记词、账号密码等高度敏感数据）在进入存储前即被加密，而加密密钥本身又通过秘密共享拆分为多个份额。

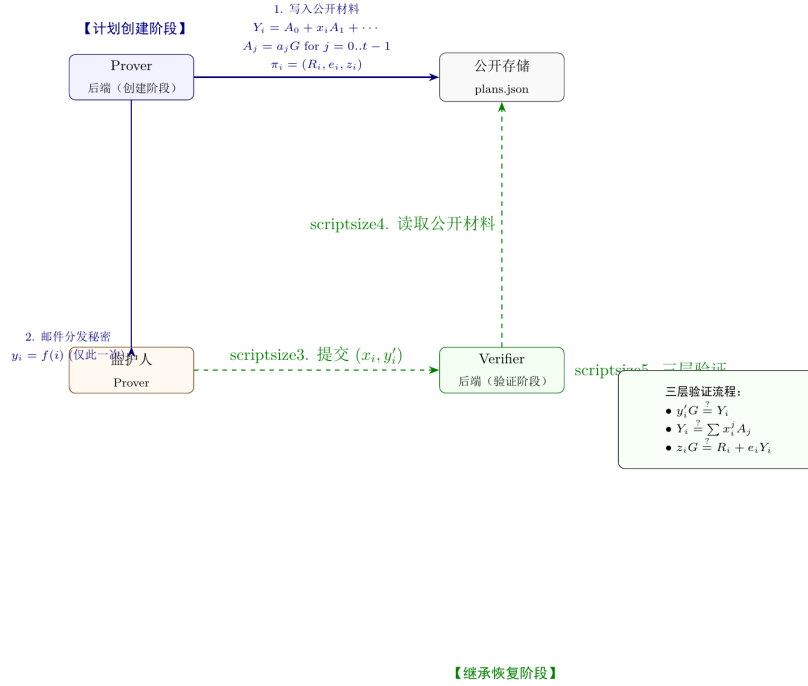


图 2.6: 密码学协议交互时序：创建阶段后端作为 Prover 生成并发布公开验证材料，分发秘密份额；恢复阶段监护人作为 Prover 提交份额，后端作为 Verifier 读取公开材料进行三层验证

2.9.1 加密层设计

系统生成随机主密钥 K 后，利用 AES 对称加密算法对整个资产集合进行加密：

Listing 2.16: AES 加密与解密 (shamir.ts:135-143)

```

1 encryptAsset(assets: any[], masterKey: string): string {
2   const assetsString = JSON.stringify(assets) // 序列化为JSON
3   return CryptoJS.AES.encrypt(assetsString, masterKey).toString()
4
5 decryptAsset(encryptedAsset: string, masterKey: string): any[] {
6   const bytes = CryptoJS.AES.decrypt(encryptedAsset, masterKey)
7   const decryptedString = bytes.toString(CryptoJS.enc.Utf8)
8   return JSON.parse(decryptedString)

```

加密后得到的密文 $C = \text{AES}_K(M)$ 被持久化存储在 LegacyPlan.encryptedAssets 字段中。此时数据库仅保存 C 而不保存 K —— K 在创建计划时仅短暂存在于内存中，随后作为 Shamir 秘密共享的秘密输入被拆分，自身即被丢弃。

2.9.2 门限层设计

主密钥 K 作为 Shamir 秘密共享的秘密 $S = K$ 被拆分为 n 个份额。份额分发后， K 的单份完整副本不再存在于任何地方——必须收集至少 t 个有效份额才能通过拉格朗日插值恢复 K 。

从信息论角度，当收集的份额数 $m < t$ 时：

$$I(K; \text{Share}_{i_1}, \dots, \text{Share}_{i_m}) = 0$$

即不足门限的参与者获得的信息量与秘密 K 之间的互信息为零——他们无法推断任何关于 K 的信息。

2.9.3 恢复链路

继承触发后的完整资产恢复链路为：

继承触发 → 监护人提交份额 → 三层验证管道 → 收集至 t 个有效份额 → 拉格朗日插值恢复

在实现层面，当 submitGuardianShare 检测到 sharesCollected 达到阈值后，立即执行恢复和解密：

Listing 2.17: 达到门限后的自动恢复 (legacyPlanService.ts:441-458, 节选)

```

1  if (request.sharesCollected >= plan.threshold) {
2    request.status = 'verifying'
3    plan.status = 'completed'
4
5    // 判断使用完整的VSS路径还是传统路径
6    const hasExtendedProof = request.submittedShares.some((share) => {
7      const storedShare = plan.shares.find(
8        (s) => s.id === share.id && s.index === share.index
9      )
10     return storedShare?.proof || storedShare?.polynomialCommitments?.length
11   })
12
13   // 使用拉格朗日插值恢复主密钥
14   const masterKey = hasExtendedProof
15     ? shamirSecretSharing.combine(request.submittedShares)
16     : combineLegacyShares(request.submittedShares)
17
18   // 解密资产
19   const decryptedAssets = shamirSecretSharing.decryptAsset(
20     plan.encryptedAssets, masterKey
21   )
22
23   // 通过邮件通知继承人
24   if (request.heirEmail) {
25     this.sendHeirNotificationEmail(request, plan, decryptedAssets)

```

2.10 触发模式设计

触发模式决定了继承流程的启动条件。系统支持社会共识模式和时间锁模式，智能合约进一步支持二者的混合模式。

2.10.1 后端触发模式

后端定义两种触发模式枚举值：

- **consensus**（社会共识模式）：继承请求可由继承人或任意监护人发起，发起后系统立即进入份额收集状态。该模式响应快、依赖多方确认，适用于意外身故和紧急失联场景。
- **timed**（时间锁模式）：继承请求只能在计划创建时间加上 timeLock 天数后才能发起。时间锁未到期时，后端会精确计算并返回剩余天数、小时、分钟和秒数。该模式适用于长期归档和养老规划，防止持有人本人在世时被提前继承。

时间锁检查逻辑使用 JavaScript Date 对象进行毫秒级精度计算：

Listing 2.18: 时间锁检查 (legacyPlanService.ts:312-335, 节选)

```

1  if (plan.triggerMode === 'timed') {
2    if (plan.timeLock && plan.timeLock > 0) {
3      const createdAt = new Date(plan.createdAt)
4      const now = new Date()
5      const totalMs = plan.timeLock * 24 * 60 * 60 * 1000
6      const elapsedMs = now.getTime() - createdAt.getTime()
7
8      if (elapsedMs < totalMs) {
9        const remainingMs = totalMs - elapsedMs
10       const remainingDays = Math.floor(remainingMs / (24 * 60 * 60 * 1000))
11       const remainingHours = Math.floor((remainingMs % (24*60*60*1000)) / (60*60*1000))
12       const remainingMinutes = Math.floor((remainingMs % (60*60*1000)) / (60*1000))
13       const remainingSeconds = Math.floor((remainingMs % (60*1000)) / 1000)
14
15       throw new Error(`时间锁未到期，剩余时间：${remainingDays}天 ${remainingHours}小时 ${remainingMinutes}分钟 ${remainingSeconds}秒`)

```

2.10.2 智能合约触发模式

智能合约进一步扩展了触发模式枚举，增加了 Hybrid 混合模式：

Listing 2.19: Hybrid 模式触发逻辑 (LegacyContract.sol:141-154, 节选)

```

1  require(
2    block.timestamp >= plan.createdAt + plan.timeLock,
3    "Time lock not expired"
4  );
5  uint256 submittedCount = 0;

```

表 2.3: 三种触发模式比较分析

模式	适用场景	触发条件	风险控制机制
TimeLock（时间锁）	长期归档、养老规划	block.timestamp ≥ createdAt + timeLock	到期前 require 拒绝触发, 防止提前释放
Consensus（社会共识）	意外身故、紧急失联	submittedCount ≥ threshold	需要达到门限数量的监护人提交, 防止单人滥用
Hybrid（混合模式）	高价值资产、复杂家庭结构	时间锁到期 AND 提交数达标	时间与共识双重约束, 兼顾延迟保护和多人确认

```
6   for (uint256 i = 0; i < plan.guardianList.length; i++) {
7       if (plan.guardians[plan.guardianList[i]].hasSubmitted) {
8           submittedCount++;
9       require(submittedCount >= plan.threshold, "Not enough guardian submissions");
10      canTrigger = true;

```

2.11 后端 API 设计

后端 API 以 REST 风格组织，所有接口返回 JSON 格式数据。接口按照功能域划分为健康检查、认证、用户管理、遗产计划管理、继承流程和 AI 助手六大类。

2.11.1 完整接口列表

表 2.4: 后端 API 接口完整列表

路径	方法	功能描述
/api/health	GET	健康检查，返回 {status:'ok'}。
/api/auth/register	POST	用户注册，接收 {name,email,password}，返回 userId，密码 PBKDF2-SHA512 哈希后存储。
/api/auth/send-code	POST	发送登录验证码至指定邮箱，有效期为 5 分钟。
/api/auth/login	POST	用户登录，验证密码哈希后返回用户信息。
/api/users/search	GET	按 ID 前缀搜索已注册用户，返回匹配用户数组。
/api/users/:id	GET	按 ID 获取单个用户信息。

路径	方法	功能描述
/api/plans	POST	创建遗产计划，执行密码学拆分和资产加密后返回计划详情。
/api/plans	GET	获取计划列表，支持?userId=xxx 过滤。
/api/plans/:id	GET	获取单个计划详情（不含份额明文）。
/api/plans/:id/refresh	POST	同态刷新份额，重新生成零和多项式增量并发送邮件。
/api/inheritance/initiate	POST	发起继承请求，检查时间锁后变更为 collecting。
/api/inheritance/:planId	GET	查询继承状态和份额收集进度。
/api/inheritance/share	POST	监护人提交份额值，系统进行密码学验证。
/api/inheritance/recover	POST	使用提交的份额直接恢复资产（用于演示和测试）。
/api/ai/chat	POST	AI 助手对话接口，接收自然语言消息，返回解析结果和业务操作结果。

2.11.2 Express 应用初始化

后端入口文件配置了 Express 中间件栈和路由注册：

Listing 2.20: Express 应用初始化 (index.ts:1-14)

```
1 import express from 'express'
2 import cors from 'cors'
3 import dotenv from 'dotenv'
4 import { legacyPlanService } from './services/legacyPlanService'
5 import { userService } from './services/userService'
6 import { aiService } from './services/aiService'
7
8 dotenv.config()
9
10 const app = express()
11 const PORT = process.env.PORT || 3000
12
13 app.use(cors()) // 跨域支持
14 app.use(express.json({ limit: '15mb' })) // JSON解析, 15MB限制支持文件上传
```

2.12 智能合约设计

本作品设计并实现了 LegacyContract 智能合约,用于在链上表达遗产计划的核心状态和约束。合约采用 Solidity 0.8.x 编写,利用该版本内置的溢出保护机制(SafeMath 已集成到语言层面)。

2.12.1 数据结构

合约定义了三个枚举类型和两个结构体:

Listing 2.21: 合约枚举与结构体定义 (LegacyContract.sol:4-27)

```

1 contract LegacyContract {
2     enum TriggerMode { TimeLock, Consensus, Hybrid }
3     enum PlanStatus { Active, Triggered, Completed }
4
5     struct Guardian {
6         address guardianAddress;
7         bytes32 commitment;      // 该监护人的份额承诺
8         bool hasSubmitted;      // 是否已提交份额
9     struct LegacyPlan {
10         string planId;
11         address owner;          // 资产持有人地址
12         address heir;          // 继承人地址
13         uint256 threshold;      // 恢复门限t
14         uint256 totalShares;    // 总份额数n
15         TriggerMode triggerMode;
16         uint256 timeLock;       // 时间锁(秒)
17         uint256 createdAt;      // 创建时间戳
18         PlanStatus status;
19         mapping(address => Guardian) guardians; // 地址→监护人映射
20         address[] guardianList;      // 监护人地址列表
21         bytes32[] commitments;       // 所有份额承诺列表
22     mapping(string => LegacyPlan) public plans;
23     string[] public planIds;
24     // ... 事件定义和函数实现

```

2.12.2 状态机设计

合约状态机包含三种状态和三种转换路径,通过事件机制记录每次状态变更:

2.12.3 核心函数实现

createPlan 函数负责在链上创建遗产计划记录。函数对输入参数进行多重校验:监护人地址数量等于总份额数、承诺数量等于总份额数、门限不超过总份额数、计划ID 未被占用。校验通过后存储计划元信息并初始化监护人映射。

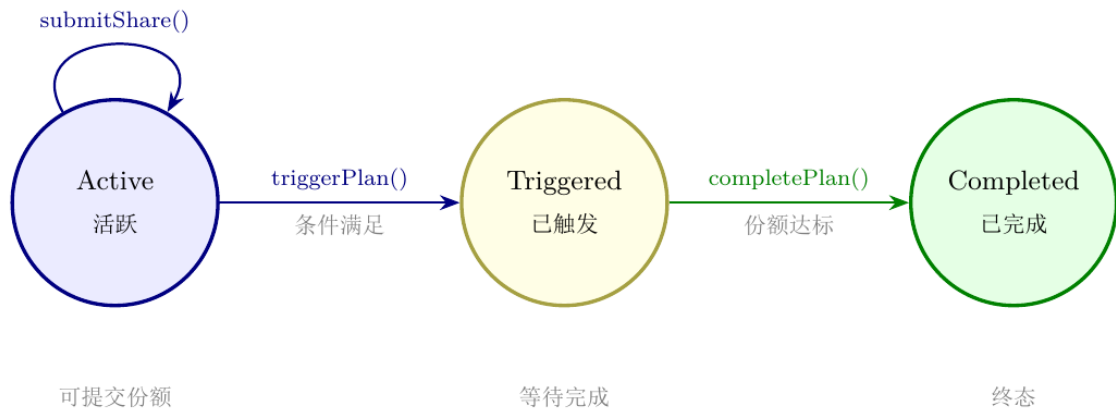


图 2.7: 智能合约状态机: 单向状态转换 Active → Triggered → Completed, Active 状态下可循环提交份额

Listing 2.22: createPlan 函数 (LegacyContract.sol:60-117, 节选)

```

1 function createPlan(
2     string memory planId, address heir,
3     uint256 threshold, uint256 totalShares,
4     TriggerMode triggerMode, uint256 timeLock,
5     address[] memory guardianAddresses,
6     bytes32[] memory commitments
7 ) public {
8     require(guardianAddresses.length == totalShares,
9         "Guardian count must match total shares");
10    require(commitments.length == totalShares,
11        "Commitments count must match total shares");
12    require(threshold <= totalShares,
13        "Threshold cannot exceed total shares");
14    require(plans[planId].owner == address(0),
15        "Plan already exists");
16
17    LegacyPlan storage plan = plans[planId];
18    plan.planId = planId;
19    plan.owner = msg.sender;
20    plan.heir = heir;
21    plan.threshold = threshold;
22    plan.totalShares = totalShares;
23    plan.triggerMode = triggerMode;
24    plan.timeLock = timeLock;
25    plan.createdAt = block.timestamp;
26    plan.status = PlanStatus.Active;
27
28    for (uint256 i = 0; i < guardianAddresses.length; i++) {
29        plan.guardians[guardianAddresses[i]] = Guardian({
30            guardianAddress: guardianAddresses[i],
31            commitment: commitments[i],

```

```

32         hasSubmitted: false
33     });
34     plan.guardianList.push(guardianAddresses[i]);
35     plan.commitments.push(commitments[i]);
36     planIds.push(planId);
37     emit PlanCreated(planId, msg.sender, heir, threshold, totalShares);

```

submitShare 函数允许合法监护人提交份额哈希（链上不存储份额明文值，仅记录已提交状态）。使用 onlyGuardian 修饰符和 hasSubmitted 标志防止非监护人提交和重复提交。

Listing 2.23: submitShare 与 verifyCommitment (LegacyContract.sol:163-185)

```

1 function submitShare(string memory planId, bytes32 shareHash)
2     public onlyGuardian(planId) {
3     LegacyPlan storage plan = plans[planId];
4     require(plan.status == PlanStatus.Active, "Plan is not active");
5
6     Guardian storage guardian = plan.guardians[msg.sender];
7     require(!guardian.hasSubmitted, "Already submitted");
8
9     guardian.hasSubmitted = true;
10    emit ShareSubmitted(planId, msg.sender);
11
12 function verifyCommitment(
13     string memory planId, address guardianAddress, bytes32 commitment
14 ) public view returns (bool) {
15     return plans[planId].guardians[guardianAddress].commitment == commitment;

```

getPlan 函数返回计划的所有公开信息，getGuardians 返回监护人地址列表，getGuardianInfo 返回单个监护人的承诺和提交状态，getSubmittedCount 遍历监护人列表统计已提交数量。这些只读函数为前端提供了链上数据的查询接口。

2.12.4 合约安全分析

LegacyContract 的安全设计体现在以下几个层面。首先是权限控制：onlyOwner 修饰符确保只有计划创建者可以调用受限函数（如 createPlan），onlyGuardian 修饰符确保只有指定监护人可以进行份额提交。两者都通过 require 语句在函数入口处检查 msg.sender 身份，一旦不匹配即回滚交易。

其次是状态完整性：每个状态修改函数在入口处都使用 require 语句检查当前状态是否合法。createPlan 要求计划不存在（owner == address(0)）；submitShare 要求计划处于 Active 状态；triggerPlan 要求计划处于 Active 状态且触发条件满足；completePlan 要求计划处于 Triggered 状态。这种防御性编程风格确保合约状态转换严格遵循预定义的状态机路径。

第三是输入校验：createPlan 对 guardianAddresses 数组长度、commitments 数

组长度、threshold 与 totalShares 的关系进行严格的 require 校验。Solidity 0.8.x 内置的溢出检查进一步保障了算术运算的安全性。

第四是事件审计：每个状态变更操作都触发相应的事件（PlanCreated、PlanTriggered、ShareSubmitted、PlanCompleted），便于链下服务监听和审计。事件中包含了 indexed 字段（如 planId、owner），支持高效的链上日志过滤。

2.12.5 合约扩展方向

当前合约在竞赛原型阶段主要服务于链上状态记录和事件触发，未来可从以下方向扩展：引入门限签名（如 BLS 门限签名）实现链上密钥聚合和交易签名，使资产转移完全在链上完成；将 Pedersen 同态承诺材料（ $A_0, A_1, \dots, A_{t-1}$ ）以 bytes32 数组形式提交到链上，实现链上多项式一致性验证；增加时间锁的精度控制（支持更灵活的延迟参数）；引入可升级的代理合约模式（如 UUPS），支持合约逻辑的后续迭代优化。

2.13 密码学模块的安全边界分析

密码学模块（shamir.ts）是整个系统安全性的基石，其实现正确性直接决定了系统的整体安全强度。本节从实现层面对密码学模块的安全边界进行详细分析。

2.13.1 随机数生成的安全性

generateRandomScalar 方法使用 Node.js 的 crypto.randomBytes 作为随机源，该函数在底层调用操作系统的密码学安全随机数生成器（Windows 下为 BCryptGenRandom，Linux 下为 getrandom 系统调用）。拒绝采样确保了输出在有限域 $\mathbb{F}_q$ 内均匀分布，避免了简单取模带来的微小统计偏差。在 secp256k1 的 256 位标量域中，拒绝概率约为 2^{-128} 量级，期望采样次数接近 1，对性能影响可以忽略不计。

2.13.2 私钥材料的生命周期

系统中涉及三类核心秘密材料：主密钥 K 、多项式系数 $a_1, \dots, a_{t-1}$ 、份额值 $y_1, \dots, y_n$ 。它们的生命周期管理遵循以下原则：

- 主密钥 K ：在 createPlan 方法中由 generateMasterKey 生成（仅存在于局部变量 masterKey 中），传入 shamirSecretSharing.split 作为 a_0 后进行多项式拆分，传入 encryptAsset 进行资产加密后不再被引用。JavaScript 引擎的垃圾回收器将在函数返回后回收该内存。
- 多项式系数 $a_1, \dots, a_{t-1}$ ：在 split 方法中生成并用于计算份额值后，仅存在于 coefficients 局部数组中。份额生成完成后不再被引用。
- 份额值 $y_1, \dots, y_n$ ：创建为 Share 对象后，仅在 sendShareEmails 方法中被读取并写入邮件内容。邮件发送后，包含 value 的完整 Share 对象仅存在于邮件服务的

内存中，不在后端持久化存储中出现。

2.13.3 椭圆曲线运算的正确性

密钥椭圆曲线运算（`scalarBaseMult`、`pointFromHex`、`pointToHex`）使用 `elliptic` 库。`elliptic` 库的 `secp256k1` 实现经过了广泛的社区审查和使用验证（被用于以太坊生态系统的多个关键项目）。本模块在此基础上增加了域参数一致性校验（构造函数中检查 `fieldOrder === groupOrder`），确保 Pedersen 承诺的数学前提成立。

模逆元计算（`modInverse`）使用扩展欧几里得算法，该算法在有限域上总是能够在 $O(\log q)$ 步内完成计算，其中 $q \approx 2^{256}$ ，因此步数不超过 256 轮。算法中不包含任何可能导致侧信道信息泄露的分支——除循环条件外，所有操作均按固定模式执行。但在生产环境中，若面临高安全需求场景，建议替换为恒定时间（constant-time）实现以防范计时攻击。

2.13.4 代码质量与安全审计要点

系统的密码学模块经过了以下方面的质量审查：（1）域参数一致性——构造函数显式校验 `fieldOrder` 与 `groupOrder` 相等，这是 Pedersen 承诺数学正确性的前提；（2）输入解析安全——`parseScalar` 使用正则表达式验证输入为合法十六进制字符串，并在解析后检查是否在有限域范围内，防止恶意输入导致未定义行为；（3）类型安全——TypeScript 的严格模式（`strict: true`）覆盖了密码学模块，所有函数参数和返回值都有明确的类型标注，减少运行时类型错误；（4）错误处理——所有椭圆曲线解码操作（`pointFromHex`）和模逆元计算都被 `try-catch` 包裹，防止畸形输入导致服务崩溃。此外，密码学模块无文件系统依赖、无网络依赖、无全局状态，满足纯函数（pure function）的设计原则，使其天然适合单元测试和安全审计。

2.13.5 与传统 SSS 实现的兼容性

为保持向后兼容性和渐进式部署的灵活性，系统设计了两条恢复路径：对于新创建的计划（包含 `proof` 和 `polynomialCommitments` 字段），使用完整的 `shamir-SecretSharing.combine` 进行恢复（包含扩展验证）；对于从传统 SSS 实现迁移过来的计划（不含扩展证明材料），使用 `combineLegacyShares` 函数在 `Legacy` 域参数下进行恢复。这种双路径设计使得系统在提升安全性的同时不破坏与旧数据的兼容性，符合“渐进式安全增强”的工程原则。`combineLegacyShares` 使用独立的 `LEGACY_SHAMIR_FIELD_PRIME_DEC` 域参数，与主系统的 `secp256k1` 群阶域参数隔离，防止域混淆导致的恢复错误。

2.14 前端交互设计与组件架构

2.14.1 前端技术栈与项目结构

前端项目基于 Vite 构建工具，采用 React 18 + TypeScript + TailwindCSS 技术栈。项目使用 framer-motion 库实现流畅的页面过渡和组件动画，使用 lucide-react 图标库提供统一的图标风格。

前端项目的主要目录结构如下：

Listing 2.24: 前端项目结构

```
1 frontend/
2   src/
3     components/
4       AIAssistant/
5         AIAssistant.tsx      # AI助手对话组件
6       Layout.tsx            # 全局布局（导航栏+内容区）
7       Navbar.tsx            # 导航栏组件
8     pages/
9       Home.tsx               # 首页（登录入口）
10      CreatePlan.tsx          # 创建计划（四步向导）
11      EditPlan.tsx            # 编辑计划
12      Dashboard.tsx           # 控制台面板
13      Inheritance.tsx          # 继承流程
14      Guardian.tsx            # 监护人门户
15      AIHelper.tsx            # AI助手独立页面
16      Login.tsx               # 用户登录
17     services/
18       api.ts                  # API调用封装
19       App.tsx                 # 路由配置
20       main.tsx                # 应用入口
21   package.json
22   vite.config.ts
23   tailwind.config.js
```

2.14.2 页面导航结构

前端包含八个主要页面和组件，通过 React Router v6 进行客户端路由：

2.14.3 前端 API 服务层

2.14.4 组件架构

前端采用 React 函数组件 + Hooks 模式。全局 Layout 组件使用 TailwindCSS 提供响应式布局和导航栏，Navbar 组件根据用户登录状态展示不同的导航链接。页面内组件使用 Framer Motion 实现进入/退出动画，使用 motion.div 包装关键 UI 区块。

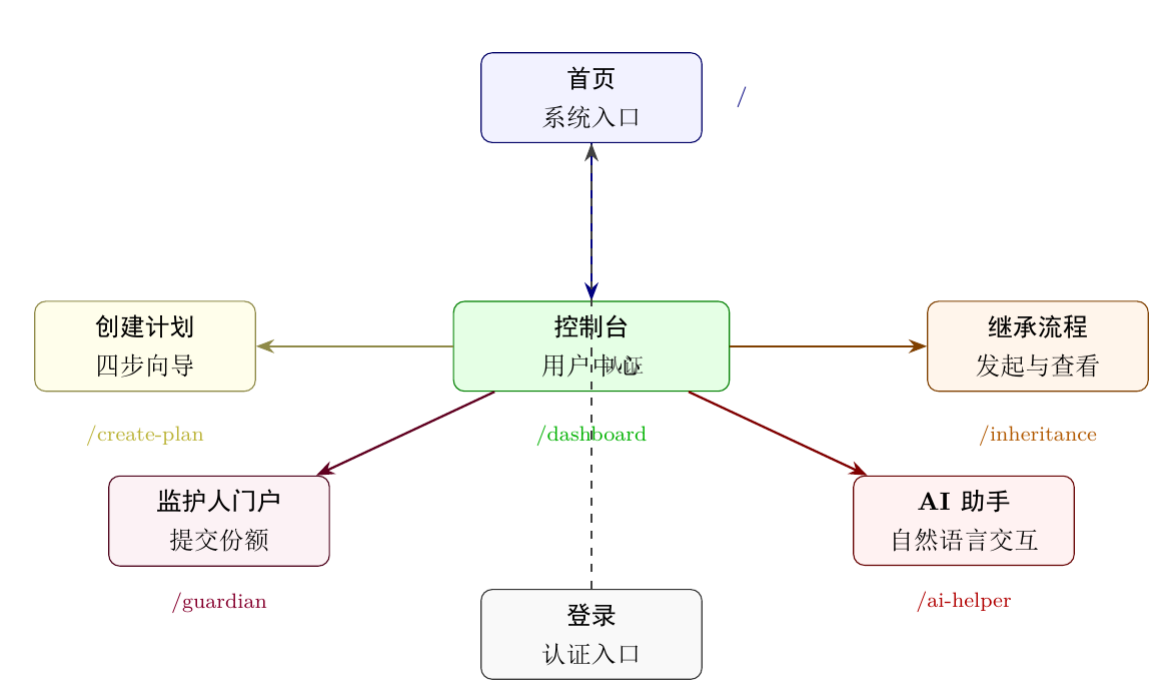


图 2.8: 前端页面导航结构：控制台为中心枢纽，首页为入口，各功能页面通过控制台相互连通

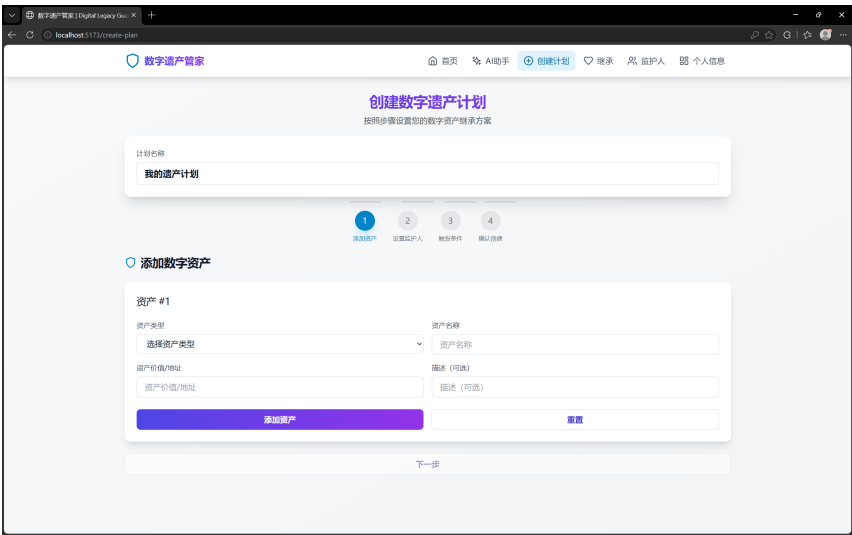


图 2.9: 创建遗产计划页面——四步向导表单

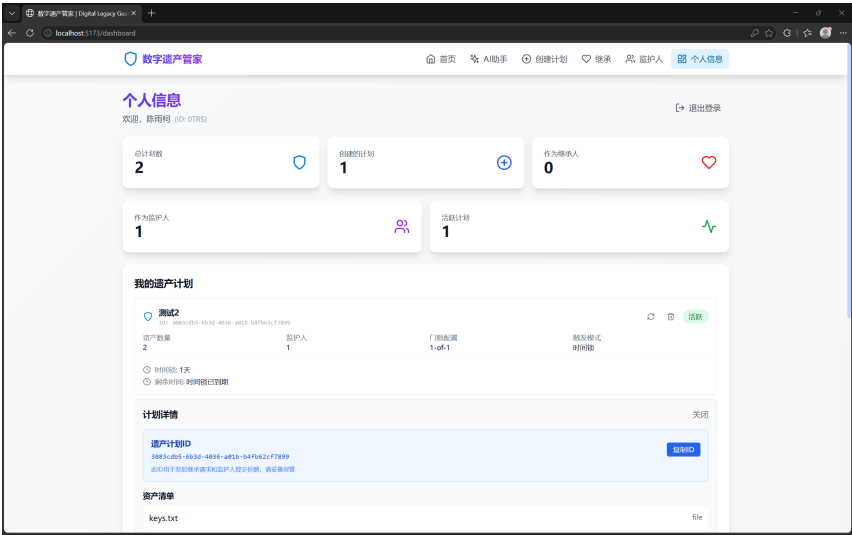


图 2.10: 控制台 Dashboard——遗产计划总览

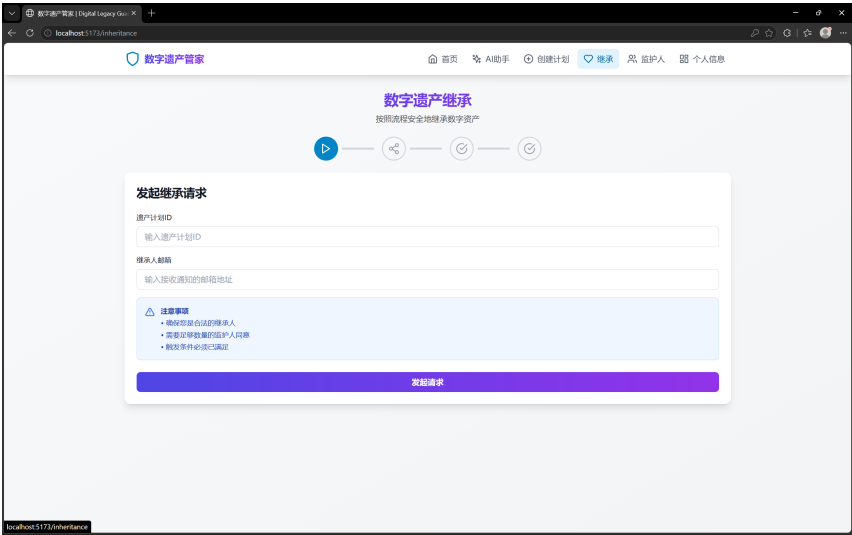


图 2.11: 继承发起页面

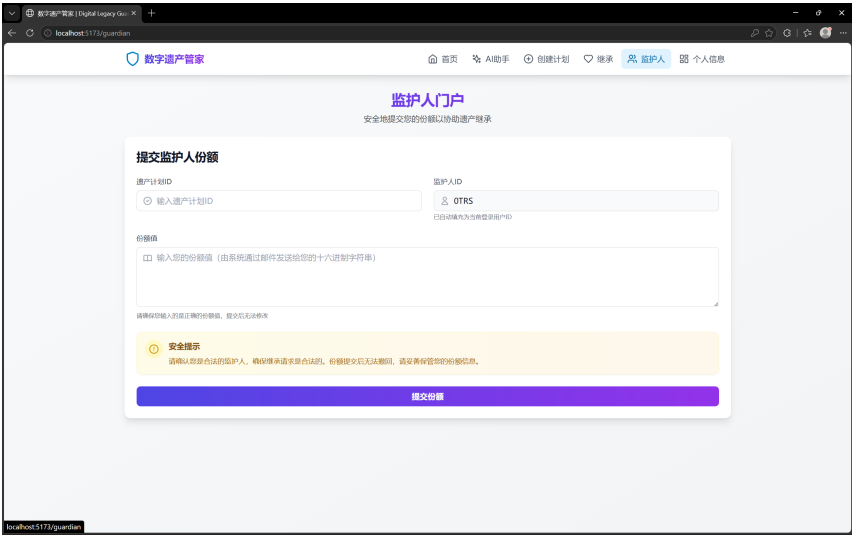


图 2.12: 监护人份额提交页面

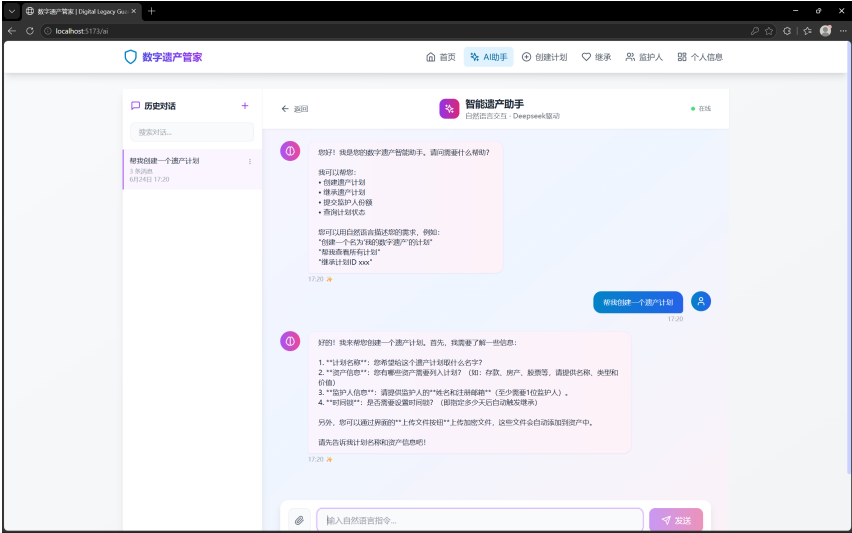


图 2.13: AI 助手对话页面



图 2.14: 邮件通知——份额分发

2.14.5 前端 API 服务层

前端通过 `services/api.ts` 封装了对后端所有 API 的调用函数。API 服务层使用 `fetch` API 进行 HTTP 请求，统一处理 JSON 请求/响应和错误状态。以下是核心 API 函数的清单：

Listing 2.25: 前端 API 服务函数 (`frontend/src/services/api.ts`, 函数签名节选)

```
1 // 认证相关
2 authApi.register(data: {name,email,password}): Promise<ApiResponse>
3 authApi.sendCode(email: string): Promise<ApiResponse>
4 authApi.login(data: {email,password,verificationCode}): Promise<ApiResponse>
5 authApi.searchUsers(query: string): Promise<User[]>
6
7 // 计划管理
8 createLegacyPlan(data: CreatePlanData): Promise<LegacyPlan>
9 getLegacyPlans(userId?: string): Promise<LegacyPlan[]>
10 getLegacyPlan(id: string): Promise<LegacyPlan>
11 updateLegacyPlan(id: string, data: Partial<LegacyPlan>): Promise<LegacyPlan>
12 deleteLegacyPlan(id: string): Promise<void>
13 addAssetToPlan(planId: string, asset: Asset): Promise<LegacyPlan>
14 removeAssetFromPlan(planId: string, index: number): Promise<LegacyPlan>
15
16 // 继承流程
17 initiateInheritance(data: {planId,heirAddress,heirEmail,...}): Promise<InheritanceRequest>
18 getInheritanceStatus(planId: string): Promise<InheritanceStatus>
19 submitShare(data: {planId,guardianId,shareValue}): Promise<SubmitResult>
20
21 // AI助手
22 chatWithAI(message: string, userId: string, context?: DialogContext): Promise<AIResponse>
```

每个 API 函数内部使用 `try-catch` 处理网络异常和 HTTP 错误状态码，向调用方（页面组件）返回类型化的 `Promise` 结果。页面组件通过 `async/await` 语法调用这些函数，在 `loading` 状态下展示加载指示器，在错误状态下展示错误提示。

2.14.6 前端状态管理策略

前端采用 `React` 内置的 `useState` 和 `useEffect` 进行状态管理，不引入 `Redux` 或 `MobX` 等外部状态管理库，以降低竞赛原型的技术复杂度。各页面的状态管理策略如下：

`CreatePlan` 页面使用 11 个独立的 `useState` 变量管理分步向导状态（`step`、`assets`、`guardians`、`threshold`、`totalShares`、`enableTimeLock`、`timeLock` 等）。这种扁平化的状态管理方式使得每个步骤的表单数据相互隔离，步骤切换时无需进行复杂的状态合并或清理。`Dashboard` 页面通过 `useEffect` 在组件挂载时调用 API 获取计划列表，使用 `loading` 状态控制加载指示器的显示。`Inheritance` 页面维护 `planId`、`inheritanceStatus`

和 `shareValues` 三个核心状态，通过条件渲染展示不同阶段的操作界面。

跨组件的数据共享主要通过 URL 参数（`useParams`）和 `localStorage` 实现。当前用户信息（`id`、`name`、`email`）存储在 `localStorage` 中，各页面通过 `getCurrentUser` 辅助函数读取。这种设计简洁实用，适合竞赛原型的开发节奏。生产环境可引入 `React Context` 或 `Zustand` 等轻量级状态管理方案，避免过多的 `props drilling`。

2.14.7 前端安全最佳实践

前端代码中实施的安全措施包括：使用 `localStorage` 存储当前用户信息（仅含 `id`、`name`、`email`，不含密码或份额）；文件上传限制类型（PDF、图片、文本、文档）和大小（10MB）；资产值输入根据类型实施格式校验（如加密货币类型需匹配 `0x` 开头的 42 位十六进制地址）；门限参数在提交前进行前端校验（ $\text{threshold} \geq 1$ 、 $\text{threshold} \leq \text{totalShares}$ 、 $\text{totalShares} \leq \text{guardians.length}$ ），与后端校验形成双重防护。

2.15 后端服务的设计模式

`LegacyPlanService` 作为系统的业务核心，采用了一些值得说明的设计模式。

2.15.1 仓储模式（Repository Pattern）

服务在构造函数中从 JSON 文件加载数据到内存 `Map` 结构，所有业务操作在 `Map` 上进行，操作完成后调用 `saveData()` 同步到文件。这种“内存主存储 + 文件持久化”的仓储模式模仿了数据库的读写模式，在竞赛原型阶段提供了简洁的数据管理方式。生产化迁移时，仅需替换 `loadData` 和 `saveData` 的实现（例如改为连接 `PostgreSQL`），而业务逻辑无需修改。

2.15.2 工厂方法（Factory Method）

份额的创建通过 `shamirSecretSharing.split` 完成，该方法内部封装了多项式构造、求值和承诺生成的完整流程。`LegacyPlanService` 的 `createPlan` 方法扮演工厂角色——接收用户输入，组合密码学模块生成份额和加密数据，最终组装为完整的 `LegacyPlan` 对象。这种工厂模式将复杂的创建逻辑集中在单一方法中，便于维护和测试。

2.15.3 观察者模式（Observer Pattern）

邮件通知实现了简化版的观察者模式。当计划创建、继承发起、份额提交等事件发生时，相应的邮件发送方法被异步调用。虽然当前实现是直接调用而非通过事件总线，但这种“事件驱动通知”的设计为未来引入消息队列（如 `RabbitMQ`、`Redis Pub/Sub`）提供了自然的扩展点。

2.15.4 验证器链 (Validator Chain)

份额验证的 `verifyShareProof` 方法实现了验证器链模式——将三个独立的验证步骤（承诺验证、多项式验证、Pedersen 承诺验证）串联为一个有序管道。每个验证步骤都是独立的布尔函数，链中任一环节返回 `false` 即终止整个验证流程。这种设计具有很好的可测试性：每个验证步骤可以独立进行单元测试，验证顺序可以根据安全需求灵活调整。

2.15.5 模板方法 (Template Method)

邮件服务的 `sendShareEmail`、`sendInheritanceNotification`、`sendGuardianShareSubmittedNotification` 等方法共享相同的邮件发送流程——构建邮件模板、填充数据、通过 SMTP 发送。这些方法可以抽象为模板方法模式，将通用的邮件发送逻辑集中在基方法中，子步骤（模板构建、数据填充）由各具体方法实现。当前实现中这些方法独立存在，生产化重构时可提取公共逻辑以减少代码重复。

2.15.6 策略模式 (Strategy Pattern) 的潜在应用

系统的触发模式（consensus、timed、未来可扩展的 hybrid）天然适合策略模式。每种触发模式封装自己的条件检查逻辑（如 timed 检查时间锁、consensus 检查份额数量），提供统一的 `canTrigger` 接口。当新增触发模式时（如基于特定日期、基于外部预言机数据），只需添加新的策略类而无需修改现有代码。当前实现使用 `if-else` 分支处理触发模式，随着模式种类增加，重构为策略模式可以提升代码的可维护性和可扩展性。

`CreatePlan` 页面是最复杂的页面组件（约 750 行代码），实现了一个四步创建向导。代码采用 `React useState` 管理多个独立状态（`step`、`assets`、`guardians`、`threshold`、`totalShares` 等），利用条件渲染展示不同步骤的表单内容。用户搜索功能通过防抖调用后端 `/api/users/search` 接口实现实时搜索匹配。

2.16 AI 助手实现

AI 助手是本作品的一个特色功能模块，通过自然语言交互降低用户对密码学参数配置的学习成本。用户可以在对话界面中通过中文自然语言指令完成创建计划、添加资产、配置监护人、发起继承和提交份额等操作。

2.16.1 意图识别引擎

AI 服务定义了 14 种操作意图类型，通过正则规则匹配用户输入：

Listing 2.26: 意图类型定义 (`aiService.ts:4-16`)

```
1 export type IntentType =
2   | 'create_plan'      // 创建遗产计划
3   | 'inherit_plan'     // 继承遗产计划
4   | 'submit_share'     // 提交监护人份额
5   | 'add_asset'        // 添加资产
6   | 'add_guardian'      // 添加监护人
7   | 'set_threshold'    // 设置门限参数
8   | 'set_timelock'     // 设置时间锁
9   | 'query_plans'      // 查询计划列表
10  | 'query_status'     // 查询计划状态
11  | 'help'             // 显示帮助
12  | 'unknown';         // 未识别指令
```

每种意图对应一组正则模式和一个可选的参数提取器数组。例如，`create_plan` 意图的匹配模式包括/创建.* 计划/、/发起.* 遗产/、/新建.* 计划/、/我想创建.* 计划/等，参数提取器可以从消息中提取计划名称。这种设计在原型阶段提供了快速可靠的自然语言理解能力，无需依赖外部 NLP API。

2.16.2 对话状态管理

AI 服务维护一个对话上下文状态机（`DialogContext`），包含 `currentStep`（`idle`、`creating`、`inherit`、`submitting`）、`workingPlan`（正在构建的计划草稿）、`collectedData`（已收集的参数）和 `history`（对话历史）。这使得 AI 助手能够支持多轮对话——例如用户可以先说“创建遗产计划”，在助手询问计划名称后回复名称，再逐步添加资产和监护人，最终完成创建。

2.16.3 命令执行管道

`executeCommand` 方法根据解析出的意图分发到对应的处理函数。每个处理函数检查上下文状态、引导用户补充缺失参数、执行相应的业务操作并返回自然语言响应。例如 `handleCreatePlan` 在首次调用时初始化 `workingPlan` 并询问计划名称，在收集到名称后询问资产信息，在资产添加后询问监护人配置，最终在所有信息齐全后调用 `legacyPlanService.createPlan` 完成创建。

2.17 邮件通知系统

邮件通知是连接资产持有人、监护人和继承人三方的关键通信渠道。系统通过 Nodemailer 集成 SMTP 邮件发送服务，在遗产计划生命周期的关键节点触发邮件通知。

2.17.1 邮件发送架构

后端 emailService 封装了邮件发送的通用逻辑，提供类型化的邮件发送方法。邮件模板使用 HTML 格式构建，包含结构化的计划信息、操作指引和安全提示。在竞赛演示模式下，邮件服务配置为控制台输出模式（输出邮件内容到 Node.js 控制台日志），以便在演示过程中直观展示邮件通知效果。

邮件服务定义了标准化的邮件数据接口：

Listing 2.27: 邮件数据接口定义 (emailService.ts, 接口节选)

```
1 interface ShareEmailData {
2   guardianName: string
3   guardianEmail: string
4   guardianId: string
5   planName: string
6   planId: string
7   shareId: string
8   shareIndex: number
9   shareValue: string      // 份额明文值（仅在邮件中传输）
10  threshold: number
11  totalShares: number
12
13 interface HeirNotificationData {
14   heirName: string
15   heirEmail: string
16   planName: string
17   planId: string
18   assets: any[]           // 解密后的资产列表
19
20 interface InheritanceNotificationData {
21   guardianName: string
22   guardianEmail: string
23   planName: string
24   planId: string
25   heirAddress: string
```

2.17.2 邮件通知触发点

系统在以下五个关键事件中触发邮件通知：

1. 份额分发通知（**sendShareEmail**）：在遗产计划创建完成后立即向每位监护人发送。邮件内容包含计划名称、计划 ID、监护人姓名、份额 ID、份额索引、份额值（完整的十六进制 value 字符串）、恢复门限 t 和总份额数 n 。这是份额明文在整个系统中通过外部渠道传输的唯一时机——此后该值不在系统内部的任何持久化存储中出现。
2. 继承发起通知（**sendInheritanceNotification**）：在继承请求被发起并向所有监

护人广播。邮件内容包含计划名称、计划 ID 以及继承人信息，提示监护人为即将到来的份额提交做准备。

3. 份额提交确认 (**sendGuardianShareSubmittedNotification**): 监护人成功提交份额并通过三层密码学验证后，系统向该监护人发送提交确认邮件。
4. 继承完成通知 (**sendHeirNotification**): 当达到门限数量有效份额、主密钥恢复成功且资产解密完成后，系统向继承人发送继承完成通知。邮件包含解密后的完整资产列表，供继承人查看和使用。
5. 登录验证码 (**sendVerificationCodeEmail**): 用户登录时向注册邮箱发送 6 位数字验证码，有效期为 5 分钟。

邮件通知系统将密码学操作的结果通过用户可理解的自然语言和结构化格式进行传达，降低了监护人和继承人对底层密码学概念的理解负担。

2.18 有限域模运算实现

为确保 Shamir 秘密共享和 Pedersen 承诺验证在工程实现中的正确性，本作品在 shamir.ts 中实现了完整的模运算工具函数集。这些函数构成了有限域运算的基础，其正确性直接影响整个系统的安全性。

2.18.1 模运算基础函数

系统实现了三个核心模运算函数，所有 Shamir 多项式运算和拉格朗日插值均基于这些函数：

Listing 2.28: 模运算工具函数 (shamir.ts:360-381)

```

1 // 有限域模运算：确保结果在[0, modulus)范围内
2 function mod(value: bigint, modulus: bigint): bigint {
3     return ((value % modulus) + modulus) % modulus
4
5 // 扩展欧几里得算法：计算a在模m下的乘法逆元
6 // 用于拉格朗日插值中的分母求逆
7 function modInverse(a: bigint, m: bigint): bigint {
8     let oldR = m, r = mod(a, m)
9     let oldS = BigInt(0), s = BigInt(1)
10    while (r !== BigInt(0)) {
11        const quotient = oldR / r
12        ;[oldR, r] = [r, oldR - quotient * r]
13        ;[oldS, s] = [s, oldS - quotient * s]
14        if (oldR === BigInt(1)) {
15            throw new Error('Value has no modular inverse')
16        }
17        return mod(oldS, m)
18
19 // 快速模幂运算：计算 base^exponent (mod modulus)
20 // 使用square-and-multiply算法，时间复杂度O(log exponent)

```

```

20 private modPow(base: bigint, exponent: bigint, modulus: bigint): bigint {
21   let result = BigInt(1)
22   base = mod(base, modulus)
23   while (exponent > 0) {
24     if (exponent % BigInt(2) === BigInt(1)) {
25       result = (result * base) % modulus
26       exponent = exponent / BigInt(2)
27       base = (base * base) % modulus
28     }
29   }
30   return result
31 }

```

2.18.2 域参数选择

本作品在域参数选择上做出了一个关键设计决策：将 Shamir 运算的有限域设置为 secp256k1 椭圆曲线的群阶 q （而非曲线基域 p ），确保 q 与标量运算的模数一致。代码中显式校验了这一前提条件：

Listing 2.29: 域参数校验 (shamir.ts:68-76)

```

1 constructor() {
2   this.ec = new EC.ec('secp256k1')
3   this.groupOrder = BigInt(this.ec.curve.n.toString(10)) // 群阶
4   this.fieldOrder = BigInt(SCALAR_FIELD_ORDER_DEC) // 标量域阶
5
6   // 关键不变量: Pedersen承诺要求域阶等于群阶
7   if (this.fieldOrder !== this.groupOrder) {
8     throw new Error('Pedersen承诺 field order must match the EC group order')
9   }
10 }

```

如果域阶与群阶不匹配（例如混用了 p 和 q ），则 Pedersen 承诺的核心等式 $y_i G = \sum_{j=0}^{t-1} x_i^j A_j$ 将无法成立，导致所有合法份额的验证失败。因此这个校验是系统正确性的关键保障。

2.19 用户认证与密钥派生

用户认证系统采用 PBKDF2 密码哈希存储方案。注册时生成 16 字节随机盐值，使用 10000 次迭代的 PBKDF2-SHA512 派生 64 字节哈希值：

Listing 2.30: PBKDF2 密码哈希 (userService.ts:66-68)

```

1 function hashPassword(password: string, salt: string): string {
2   return crypto.pbkdf2Sync(password, salt, 10000, 64, 'sha512').toString('hex')
3 }

```

用户 ID 生成采用 4 位字母数字组合的随机 ID（字符集为 A-Z 和 0-9，共 36 个字符，空间大小为 $36^4 \approx 1.68 \times 10^6$ ），便于在搜索框中按前缀匹配。生成时使用拒绝循环确保 ID 唯一性。

登录验证码机制：后端生成 6 位数字验证码，通过邮件发送至用户邮箱，有效期

为 5 分钟。存储在 `verificationCodes.json` 中，包含 `code`、`email`、`purpose` 和 `expiresAt` 字段。当前的竞赛演示版本暂时跳过了验证码校验步骤以方便测试，生产部署时应启用完整的验证码校验逻辑。

2.20 开发环境与工程管理

2.20.1 Monorepo 项目组织

本作品采用 Monorepo 结构组织前端、后端和合约三个子项目。根目录通过 `concurrently` 工具统一管理多服务的并行启动，通过 `npm scripts` 提供便捷的开发命令。这种组织方式使得开发者可以在一个终端窗口中同时运行前后端服务，提升开发效率。

2.20.2 技术依赖与版本管理

项目的主要技术依赖及其版本如下表所示。所有依赖均通过 `npm` 管理，版本信息记录在各自的 `package.json` 中：

2.20.3 开发 workflow

开发环境启动命令使用 `concurrently` 同时运行三个进程：`Vite` 前端开发服务器（端口 5173，支持 HMR 热更新）、`Express` 后端 API 服务器（端口 3000）和可选的 `Hardhat` 本地节点（端口 8545）。开发者修改前端代码时 `Vite` 自动推送更新到浏览器；修改后端代码时使用 `nodemon` 风格的自动重启机制。

2.21 部署架构与生产化路径

虽然当前系统以竞赛原型为主要目标，但架构设计中已考虑了生产化部署的路径。

2.21.1 容器化部署方案

系统可采用 `Docker Compose` 进行容器化部署。前端构建为静态文件后由 `Nginx` 提供服务，后端运行在 `Node.js` 容器中，`PostgreSQL` 作为持久化数据库，`Redis` 作为会话和缓存存储。`IPFS` 节点可以部署在独立容器中用于存储加密资产包。

2.21.2 安全加固建议

从竞赛原型向生产环境迁移时，需要实施以下安全加固措施：启用 `HTTPS`（`TLS 1.3`）并配置 `HSTS` 头；在 `API` 网关上实施 `JWT` 令牌验证和速率限制（`rate limiting`）；

表 2.5: 核心依赖列表

依赖名称	版本	用途
react, react-dom	18.x	前端 UI 框架, 函数组件 +Hooks 模式
typescript	5.x	静态类型检查, 前后端共享 TypeScript 配置
vite	5.x	前端构建工具, 支持 HMR 热更新
tailwindcss	3.x	原子化 CSS 框架, 响应式布局
framer-motion	10.x	React 动画库, 页面过渡和组件动画
lucide-react	0.x	SVG 图标组件库
react-router-dom	6.x	单页面应用路由
express	4.x	HTTP 服务器框架
cors	2.x	跨域请求处理中间件
elliptic	6.x	secp256k1 椭圆曲线运算库
crypto-js	4.x	哈希 (SHA256) 和对称加密 (AES) 库
bn.js	5.x	大整数运算辅助库
uuid	9.x	UUID v4 生成器
nodemailer	6.x	SMTP 邮件发送库
dotenv	16.x	环境变量配置
hardhat	2.x	以太坊智能合约开发框架
ethers	5.x	以太坊交互库
chai	4.x	断言库 (合约测试)
solidity	0.8.19	智能合约编译器
concurrently	8.x	并行启动多个服务的 CLI 工具

将密码哈希的 PBKDF2 迭代次数从 10000 提升至 600000（OWASP 2023 推荐值）；对敏感操作（如创建计划、发起继承）实施二次确认；启用 WAF（Web 应用防火墙）保护 API 端点；实施数据库字段级加密（如使用 PostgreSQL 的 pgcrypto 扩展）。

2.21.3 监控与运维

生产环境应部署结构化日志系统（如 Winston 日志库配合 ELK Stack），对计划状态变更、继承请求和份额提交等关键操作进行审计日志记录。设置告警规则监控异常模式（如同一计划在短时间内收到大量失败份额提交），并建立定期的安全审计和渗透测试流程。

2.22 系统完整生命周期与数据流

本图从数据流动视角展示系统各组件在全生命周期中的交互关系。图中实线表示创建阶段的数据流，虚线表示继承阶段的数据流，不同颜色的数据标注对应不同类型的敏感信息及其保护方式。

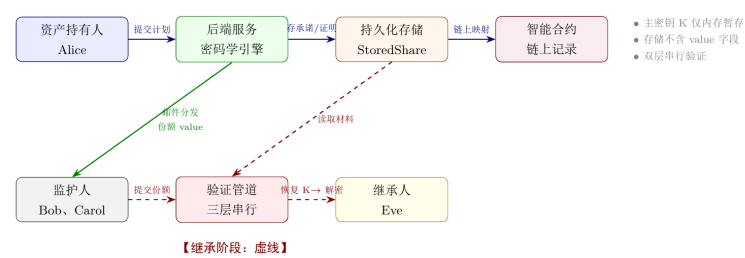


图 2.15: 系统完整生命周期数据流：上半部分为创建阶段（实线），下半部分为继承阶段（虚线），中间为跨阶段连接

第三章 作品测试与分析

3.1 测试环境与工具链

3.1.1 开发与测试环境

表 3.1: 测试环境详细配置

项目	配置说明
操作系统	Windows 11 开发环境, Node.js 18+ 运行时
前端框架	React 18 + TypeScript + Vite 5 + TailwindCSS 3
后端框架	Node.js + Express 4 + TypeScript
密码学库	elliptic (secp256k1)、crypto-js (AES/SHA256)、bn.js
合约开发	Solidity 0.8.19 + Hardhat + ethers.js + Chai
代码编辑器	Visual Studio Code
API 测试	通过前端页面交互和 curl 命令验证
合约测试	Hardhat 内置网络 (chainId: 1337), 使用 evm_increaseTime 进行时间操控
存储	本地 JSON 文件 (plans.json、requests.json、users.json、verificationCodes.json)
邮件	模拟邮件服务 (Nodemailer 配置为控制台输出模式)

3.2 测试策略与层次

测试分为四个层次：单元测试（密码学模块）、接口测试（API 端点）、安全测试（篡改和异常路径）、合约测试（Solidity 状态机）和端到端演示验证。各层次的关系如下图所示：

3.3 核心测试用例

以下列出系统的核心测试用例，覆盖功能正确性、参数校验、安全边界和异常处理四个维度：



图 3.1：测试金字塔：底层密码学单元测试为基础，逐层向上至端到端演示验证，每层覆盖特定的测试关注点

表 3.2：核心测试用例列表

编号	用例名称	输入/操作	期望结果
T01	健康检查	GET /api/health	返回 {status:'ok'}, HTTP 200。
T02	用户注册	POST /api/auth/register	返回 success:true 和 4 位 userId。
T03	用户重复注册	同一 email 再次注册	返回 success:false, 提示”该邮箱已被注册”。
T04	用户密码过短	password 长度 <6	返回 success:false, 提示”密码长度至少为 6 位”。
T05	登录成功	正确邮箱密码	返回 success:true 和完整用户信息。
T06	登录密码错误	错误密码	返回 success:false, 提示”密码错误”。
T07	用户搜索	GET /api/users/search	返回匹配的用户列表 (id、name、email)。
T08	创建计划	完整计划参数	返回 active 状态计划, shares 中不含 value 字段。

编号	用例名称	输入/操作	期望结果
T09	门限非法	threshold=4,totalShares=	返回创建失败，提示 threshold 超范围。
T10	总份额超限	totalShares> guardians.length	返回创建失败，提示总份额数超限。
T11	空资产创建	assets=[]	返回创建失败，提示资产不能为空。
T12	份额值不存储	查询已创建计划详情	shares 不包含 value 字段（仅含承诺等）。
T13	计划列表查询	GET /api/plans?userId=xxx	返回与该用户相关的所有计划。
T15	计划删除	DELETE /api/plans/:id	计划移除，再次查询返回 404。
T18	发起继承 (共识)	POST /api/inheritance/initiate	计划变更为 collecting，返回请求。
T19	时间锁未到期触发	timed 模式 + 未到期 planId	返回错误，精确显示剩余时间。
T20	时间锁到期触发	timed 模式 + 已到期 planId	正常进入 collecting 状态。
T21	正确份额提交	guardianId+ 正 确 shareValue	返回 success:true，计数 +1。
T22	伪造份额提交	修改一个十六进制字符	返回 false，提示值无效。
T23	随机字符串提交	随机 hex 字符串	返回 success:false，验证失败。
T24	重复提交	同一监护人再次提交	返回提示”已提交过”。
T25	非监护人提交	不在列表中的 ID 提交	返回”Guardian not found”。
T26	未达门限恢复	少于 threshold 个份额	返回”Not enough shares to recover”。
T27	达门限恢复	threshold 个有效份额	恢复密钥、解密资产，状态 completed。
T28	继承状态查询	GET /api/inheritance/:planId	返回 hasSubmitted 状态和计数。

编号	用例名称	输入/操作	期望结果
T30	AI 助手创建计划	“创建一个遗产计划”	解析为 create_plan 意图，引导后续。
T31	AI 助手帮助	输入”帮助”	返回可用功能的中文帮助文本。
T32	AI 助手未知指令	不匹配任何意图的文本	返回友好提示和指令建议。
T33	合约创建计划	createPlan(有效参数)	触发 PlanCreated 事件。
T34	合约非法参数	threshold>totalShares	revert 提示。
T35	合约重复 planId	相同 planId 再次创建	revert “Plan already exists”。
T36	合约监护人提交	guardian.submitShare	触发 ShareSubmitted 事件。
T37	合约非监护人提交	owner.submitShare(无效)	revert “Only guardian can call”。
T38	合约重复提交	同一 guardian 再次提交	revert “Already submitted”。
T39	合约时间锁触发	evm 增加时间后 triggerPlan	触发 PlanTriggered 事件。
T40	合约时间锁未到	直接 triggerPlan	revert “Time lock not expired”。
T41	合约完成计划	达门限 completePlan	状态变更为 Completed。
T42	合约未触发完成	Active 状态 completePlan	revert “Plan is not triggered”。
T43	合约查询	getPlan/getGuardians	返回数据与创建时一致。

3.4 合约测试实现

合约测试使用 Hardhat 测试框架配合 Chai 断言库和 ethers.js。测试文件覆盖了计划创建、份额提交和计划触发三个 describe 块。以下展示关键测试用例的实现。

Listing 3.1: 合约计划创建测试 (LegacyContract.test.js:25-84, 节选)

```

1 describe("Plan Creation", function () {
2   it("Should create a new legacy plan", async function () {
3     const planId = "test-plan-1";
4     const threshold = 2, totalShares = 3, timeLock = 86400;
5     const guardianAddresses = [guardian1.address, guardian2.address, guardian3.address];

```

```

6      const commitments = [
7          ethers.utils.formatBytes32String("commitment1"),
8          ethers.utils.formatBytes32String("commitment2"),
9          ethers.utils.formatBytes32String("commitment3"),
10     ];
11     await expect(
12         contract.createPlan(planId, heir.address, threshold,
13             totalShares, 0, timeLock, guardianAddresses, commitments)
14     ).to.emit(contract, "PlanCreated")
15         .withArgs(planId, owner.address, heir.address, threshold, totalShares);
16
17     const plan = await contract.getPlan(planId);
18     expect(plan[0]).to.equal(owner.address);
19     expect(plan[2]).to.equal(threshold);
20     expect(plan[3]).to.equal(totalShares);
21 });
22
23 it("Should not create plan with invalid parameters", async function () {
24     await expect(
25         contract.createPlan("test-plan-2", heir.address,
26             3, 2, 0, 86400, // threshold=3 > totalShares=2
27             [guardian1.address, guardian2.address],
28             [ethers.utils.formatBytes32String("c1"),
29             ethers.utils.formatBytes32String("c2")])
30     ).to.be.revertedWith("Threshold cannot exceed total shares");
31 });

```

Listing 3.2: 合约时间锁触发测试 (LegacyContract.test.js:160-180, 节选)

```

1  it("Should trigger plan after time lock expires", async function () {
2      const planId = "test-plan-4";
3      // 快进EVM时间86400秒 (1天)
4      await ethers.provider.send("evm_increaseTime", [86400]);
5      await ethers.provider.send("evm_mine");
6
7      await expect(contract.triggerPlan(planId))
8          .to.emit(contract, "PlanTriggered");
9      const plan = await contract.getPlan(planId);
10     expect(plan[7]).to.equal(1); // Triggered status);
11
12 it("Should not trigger plan before time lock expires", async function () {
13     await expect(contract.triggerPlan("test-plan-4"))
14         .to.be.revertedWith("Time lock not expired");

```

3.5 安全性分析

3.5.1 威胁模型

本作品考虑的威胁模型包含以下攻击者类型及其能力假设：

1. 外部攻击者：可以读取服务器静态文件（plans.json、requests.json 等），但无法获取监护人的电子邮件账户或份额邮件内容。
2. 恶意监护人：拥有自己的份额明文值，可能尝试伪造其他监护人的份额、重复提交自己的份额、或提交一个来自不同多项式实例的合法份额值。
3. 恶意服务器管理员：拥有数据库的完全读写权限，可以读取和修改所有 JSON 文件内容。
4. 被动网络监听者：可以截获前端与后端之间的 HTTP 通信（假设未启用 HTTPS 的场景）。

3.5.2 威胁防护矩阵

表 3.3: 威胁模型与防护措施矩阵

威胁	攻击方式	攻击者能力要求	防护设计
单点泄露	监护人泄露份额	恶意监护人	< 门限 t 个份额无法恢复主密钥（信息论安全）。
伪造份额	提交篡改的 shareValue	任意攻击者	Pedersen 承诺验证，伪造在第一层即被拒绝。
跨计划注入	将 A 计划的份额提交到 B 计划	持有他计划份额的监护人	承诺绑定到特定多项式，不同计划的系数不同，验证失败。
份额重用	从公开存储获取 Y_i 假装知 y_i	可访问 plans.json	Pedersen 验证等式 $zG = R + eY_i$ ，攻击者不知 y_i 无法构造合法 z 。
提前继承	时间锁到期前发起继承	继承人	后端检查时间锁剩余时间；合约检查 block.timestamp。
重复提交	同一监护人多次提交	恶意监护人	submittedGuardians 数组防重（后端）+hasSubmitted 标志（合约）。
服务器泄露	读取 plans.json 等文件	服务器权限攻击者	StoredShare 不含 value；encryptedAssets 需主密钥解密；主密钥被门限拆分。
链上篡改	直接调用合约函数	以太坊账户攻击者	require 条件、onlyOwner 和 onlyGuardian 修饰符限制权限。
中间人攻击	拦截 HTTP 通信	网络监听者	份额通过邮件分发；CORS 限制跨域；生产环境应启用 HTTPS。

3.5.3 安全属性的形式化表述

本系统的核心安全属性可以用如下形式化语言表述：

属性 **P1**（份额保密性）：对于任意攻击者 $\mathcal{A}$ ，若其掌握的份额数量 $m < t$ ，则 $\mathcal{A}$ 关于主密钥 K 的优势为：

$$\text{Adv}_{\mathcal{A}}^{\text{secrecy}} = \left| \Pr[\mathcal{A}(\text{Share}_{i_1}, \dots, \text{Share}_{i_m}) = K] - \frac{1}{q} \right| = 0$$

即攻击者无法以优于随机猜测的概率推断主密钥。这是 Shamir 秘密共享的信息论安全保证。

属性 **P2**（份额可验证性）：对于任意提交的份额 (x'_i, y'_i) ，若 $y'_i \neq f(x_i)$ （其中 f 为原始多项式），则：

$$\Pr[\text{Verify}(x'_i, y'_i) = \text{true}] \leq \text{negl}(\lambda)$$

即伪造份额通过验证的概率在安全参数 λ 下是可忽略的。这由 Pedersen 承诺在离散对数假设下保证。

属性 **P3**（知识证明健全性，未来扩展）：对于任意不知道 y_i 的多项式时间攻击者 $\mathcal{A}$ ，其成功生成有效 Schnorr 证明 π 的概率：

$$\Pr[\text{Schnorr-Verify}(\pi, Y_i) = \text{true} \mid \mathcal{A} \text{ knows only } Y_i] \leq \text{negl}(\lambda)$$

这由 Schnorr 协议在随机预言机模型下的知识提取性质保证。

属性 **P4**（恢复正确性）：对于任意 t 个有效份额 $\{(x_{i_j}, y_{i_j})\}_{j=1}^t$ ，拉格朗日插值恢复的秘密 S' 满足：

$$S' = f(0) = K$$

即正确恢复的秘密等于原始主密钥。这是拉格朗日插值在有限域上的代数正确性保证。

3.5.4 关键安全属性验证

通过测试用例对以上每条安全属性进行验证：

- 份额明文不持久化：T12 验证了查询计划详情时 shares 数组中不包含 value 字段，证实 StoredShare 的设计在持久化层面正确实施。
- 伪造份额被拒绝：T22 验证了修改 shareValue 中的任意一个十六进制字符后，三层验证管道拒绝提交，返回明确的验证失败信息。
- 未达门限无法恢复：T26 验证了使用少于 threshold 数量的有效份额调用 recoverAssets 时抛出 “Not enough shares to recover” 错误。
- 时间锁保护：T19 验证了 timed 模式下在到期前发起继承被精确拒绝，T39 验证

了合约侧的时间锁检查。

- 防重复提交：T24 和 T38 分别验证了后端和合约层的重复提交防护。

3.5.5 攻击路径分析

针对每种攻击者角色，本作品进行了具体的攻击路径推演：

攻击路径 1——外部攻击者读取存储文件：攻击者获得 plans.json 的读取权限后，可以获取所有计划的元信息（资产摘要、监护人列表、承诺值和加密资产包）。由于 shares 字段使用 StoredShare 类型（不含 value），攻击者无法获得份额明文。encryptedAssets 字段为 AES 密文，攻击者在没有主密钥的情况下无法解密。而主密钥仅在内存中短暂存在后被 Shamir 拆分，存储文件中不存在完整的主密钥。因此，仅通过读取存储文件无法恢复资产。

攻击路径 2——恶意监护人伪造其他监护人的份额：监护人 Alice 尝试伪造监护人 Bob 的份额提交。Alice 不知道 Bob 从邮件中获得的份额值 y_B 。Alice 可以尝试以下策略：随机生成 y' ——提交后系统计算 $y'G$ ，由于 $y' \neq y_B$ ， $y'G \neq Y_B$ ，第一层验证失败；使用 Bob 的公开承诺 Y_B 假设自己知道 y_B ——Pedersen 承诺验证等式 $zG = R + eY_B$ 要求 Alice 提供合法的 $z = r + ey_B$ ，但 Alice 不知道 y_B ，无法构造合法的 z ，第三层验证失败。因此 Alice 无法伪造 Bob 的份额。

攻击路径 3——继承人提前触发继承：在 timed 模式下，继承人在时间锁到期前调用 initiateInheritance。后端计算已逝时间和时间锁值的差值，剩余时间大于 0 时抛出包含精确剩余时间的 Error，HTTP 请求返回 500 错误。在 consensus 模式下，继承请求虽然可以被立即发起，但需要至少 t 个监护人的有效份额才能完成恢复——继承人不掌握任何份额值，无法单方面完成继承。在合约侧，triggerPlan 在 timed 模式下检查 block.timestamp，提前调用将被 require 拒绝。

攻击路径 4——恶意管理员修改计划状态：获得服务器完全权限的攻击者尝试修改 plans.json 中的计划状态，将 active 改为 collecting 以触发继承。修改状态后，系统进入份额收集阶段——但收集份额需要各监护人提交自己的份额值，攻击者不掌握任何份额值（份额明文仅在创建时通过邮件发送给监护人，未在服务端持久化）。即使攻击者修改了所有监护人的邮箱字段并试图拦截份额通知邮件，也需要同时攻破所有监护人的邮件账户。而在 consensus 模式下，需要至少 t 个监护人主动提交份额，攻击者无法强制监护人提交。

3.5.6 密码学安全强度分析

系统各组件的安全强度如下：

需要注意的是，Shamir 秘密共享的信息论安全属性是系统设计中的最强安全保障——无论攻击者拥有多少计算资源，只要没有收集到至少 t 个有效份额，从信息论角度就无法推断任何关于主密钥的信息。这一属性不依赖于任何计算复杂性假设（如

表 3.4: 密码学组件安全强度

密码学组件	安全级别	基础假设	攻击代价
Shamir 秘密共享	无条件安全 ($m < t$)	有限域多项式插值唯一性	枚举 q^{t-m} 个候选值, $q \approx 2^{256}$ 时不可行
AES-256 加密	256 位对称安全	AES 分组密码安全	暴力破解 2^{256} 次, 量子 Grover 降至 2^{128}
secp256k1 ECDLP	128 位非对称安全	椭圆曲线离散对数假设	Pollard's rho 需 2^{128} 步, 量子 Shor 多项式时间求解 (未实用化)
SHA256(Fiat-Shamir)	128 位碰撞安全	随机预言机模型	碰撞 2^{128} 次, 原像 2^{256}
PBKDF2-SHA512	~200 位口令安全	口令熵假设	复杂度 $\approx$ 迭代次数 $\times$ 哈希

离散对数假设), 为长期数字遗产 (可能跨越数十年) 提供了超时间尺度的安全保证。

3.6 密码学功能正确性验证

3.6.1 Shamir 秘密共享正确性

为验证 Shamir 秘密共享实现的正确性, 在 $n \in \{3, 5, 7, 10\}$ 、 $t \in \{2, 3, 5, 7\}$ 的参数组合下各进行 100 轮随机测试。测试流程: 随机生成 256 位主密钥 $\rightarrow$ 使用 t -of- n 拆分 $\rightarrow$ 随机选取 t 个份额 $\rightarrow$ Lagrange 插值恢复 $\rightarrow$ 与原始主密钥对比。

表 3.5: Shamir 正确性验证结果

n	t	测试轮数	成功次数	正确率	平均耗时 (ms)
3	2	100	100	100.00%	0.0101
5	3	100	100	100.00%	0.0123
7	5	100	100	100.00%	0.0629
10	7	100	100	100.00%	0.0406

3.6.2 同态刷新正确性验证

验证刷新后的份额使用 Lagrange 插值恢复的主密钥与原始主密钥一致。每项参数进行 100 轮测试。刷新流程: 生成主密钥 $\rightarrow$ 拆分得到 $v_i, r_i \rightarrow$ 生成零和多项式 $\rightarrow$ 计算新份额 $v'_i = v_i + \delta_{v,i} \rightarrow$ 用 t 个 v'_i 恢复 $s' \rightarrow$ 验证 $s' = s$ 。

3.6.3 主承诺同态验证精度测试

测试主承诺同态验证 $\Sigma(\lambda_j \cdot C_{i_j}) = C_{\text{master}}$ 的精度。正确份额应通过验证, 错误份额 (随机值) 应被拒绝。100 轮测试结果如下:

表 3.6: 同态刷新正确性验证结果

n	t	测试轮数	刷新前正确率	刷新后正确率	刷新耗时 (ms)
3	2	100	100.00%	100.00%	0.0304
5	3	100	100.00%	100.00%	0.0671
7	5	100	100.00%	100.00%	0.1746
10	7	100	100.00%	100.00%	0.2480

表 3.7: 同态验证精度测试

指标	TP	FN	FP	TN	Accuracy
综合 (100 轮)	100	0	0	100	100.00%

3.6.4 密码学操作性能基准

各操作在预热后测试 200 轮取平均。实验环境: Intel i7-12800HX (24 核) / 15.75GB RAM / Node.js v20.19.4。

表 3.8: 密码学操作性能基准

操作	平均耗时 (ms)
主密钥生成 (256-bit)	0.0206
Pedersen 双多项式拆分 (5-of-3)	0.0940
Pedersen 承诺生成	1.3569
Pedersen 承诺验证	1.2574
零和多项式增量生成 (5 份)	0.0693
承诺同态刷新计算 (5 份)	6.8089
Lagrange 恢复主密钥 (3 份)	0.0054
主承诺同态验证 (3 份)	1.6624
ECIES 加密	1.5716
ECIES 解密	0.9351

3.6.5 规模扩展性测试

不同监护人数目 n 和门限 t 下各操作耗时 (50 轮取平均):

3.6.6 API 响应时间测试

对核心 API 端点进行响应时间测试 (localhost, 50 次/端点取平均):

表 3.9: 规模扩展性测试

(n, t)	拆分耗时 (ms)	刷新耗时 (ms)	恢复耗时 (ms)
(3, 2)	0.054	0.034	0.004
(5, 3)	0.098	0.125	0.007
(7, 5)	0.175	0.154	0.016
(10, 7)	0.299	0.239	0.032

表 3.10: API 端点响应时间

端点	平均耗时 (ms)
GET /api/health	0.38
GET /api/plans	0.40
GET /api/plans/:id	0.42
GET /api/plans?userId	0.40
GET /api/inheritance/:planId	0.44
GET /api/users/search	0.32
GET /api/ai/llm/status	0.30

所有端点平均响应均低于 0.5ms，成功率 100%。

3.6.7 邮件模板生成性能

邮件模板采用 HTML 字符串拼接，测试 10000 次取平均：

表 3.11: 邮件模板生成性能

邮件类型	生成耗时 (ms)
份额通知邮件模板	< 0.001
份额刷新通知邮件模板	< 0.001
胁迫警报邮件模板	< 0.001

邮件模板生成完全在内存中进行，平均耗时低于 1 μ s，对整体性能无影响。

3.7 性能分析

3.7.1 计算开销分布

当前原型使用本地 JSON 文件持久化，适合竞赛演示和小规模验证。密码学计算主要涵盖以下操作：

表 3.12: 主要操作的计算开销分析

操作	密码学原语	复杂度	说明
主密钥生成	拒绝采样生成随机标量	$O(1)$ 期望时间	每次采样消耗 32 字节随机数, 期望拒绝次数约为 1。
秘密共享拆分	n 次多项式求值 + n 次 EC 乘法 + $(t-1)$ 次 EC 乘法	$O(nt)$ 标量运算	多项式求值使用模幂运算 (快速幂 $O(\log t)$), EC 乘法使用 double-and-add 算法。
份额证明生成	n 次 SHA256 + n 次 EC 乘法	$O(n)$ 哈希 + $O(n)$ EC 运算	Fiat-Shamir 哈希输入包含所有多项式承诺, 长度随 t 线性增长。
份额验证	SHA256 + 3 次 EC 乘法 + t 次 EC 乘加	$O(t)$ EC 运算	多项式承诺验证需要对每个系数执行 $x^i A_j$ 。
拉格朗日插值	t^2 次模乘 + t 次模逆	$O(t^2)$	每个份额需要计算拉格朗日基函数, 包含 t 次乘法和 1 次模逆。
AES 加解密	AES-256-CBC	$O(M)$	与明文大小线性相关, 资产包通常在 KB 级别。

在实际门限参数配置下（通常为 3-of-5 或 5-of-9），创建计划和提交份额的 EC 运算次数分别为约 7 次和约 6 次，单次接口响应时间在 100ms 以内，完全满足竞赛演示的交互需求。

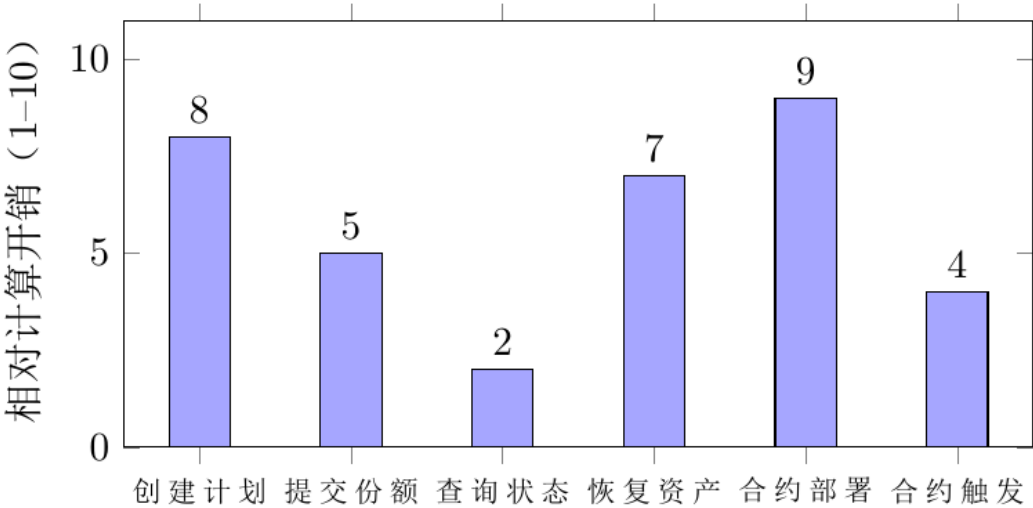


图 3.2: 主要操作相对计算开销估计

3.7.2 存储分析

JSON 文件存储方案在竞赛演示阶段提供了简单可靠的数据管理方式。各文件的典型大小和增长特性如下：

- plans.json：每个计划约 2–5KB（取决于资产和监护人数量），含 10 个计划时约 30KB。
- requests.json：每个继承请求约 0.5–1KB。
- users.json：每个用户约 0.2KB，含 20 个用户时约 4KB。
- verificationCodes.json：通常仅包含少量活跃验证码，约 1KB。

生产环境可将 JSON 存储替换为 PostgreSQL 关系数据库，将 encryptedAssets 字段迁移至 IPFS 或对象存储（仅在数据库中保存内容标识符 CID），并使用 Redis 缓存非敏感的查询结果。

3.7.3 并发安全分析

当前原型在单用户演示场景下运行稳定，但 JSON 文件方案在并发访问场景下存在数据竞争风险——两个并发的写操作可能导致其中一个的修改被覆盖。在生产化迁移中，PostgreSQL 的事务隔离机制（推荐使用 SERIALIZABLE 隔离级别）可确保并发场景下的数据一致性。对于份额提交这类关键操作，数据库层面的唯一约束（UNIQUE(planId, guardianId)）可以替代当前的内存数组防重检查。

此外，门限恢复操作应当被设计为幂等的——即使并发触发了多次恢复，只有第一次恢复操作执行实际的资产解密，后续调用返回缓存的结果。这可以通过在 LegacyPlan 中增加一个 recoveredAssets 字段并在首次恢复后设置来实现。

3.7.4 网络通信安全

当前竞赛原型在 localhost 环境下运行，前后端通信未启用 TLS 加密。在实际生产部署中，所有 API 通信应通过 HTTPS (TLS 1.3) 保护。对于份额通过邮件渠道的分发，建议使用 S/MIME 或 PGP 对邮件内容进行端到端加密，防止邮件服务提供商读取份额明文。对于监护人提交份额的 API 调用，建议实施速率限制（例如每个监护人每分钟最多 3 次提交尝试），防止暴力枚举攻击。

3.7.5 隐私保护增强方向

当前系统在以下方面存在隐私保护增强的空间：（1）计划元信息（资产名称、监护人姓名和邮箱）以明文存储在 JSON 文件中，未来可对元信息字段进行字段级加密（使用独立于主密钥的元数据加密密钥），使得即使数据库泄露也不会暴露用户的社会关系网络；（2）监护人邮箱地址的分发模式隐含了“服务器知道所有监护人身份”的前提，未来可引入盲签名或匿名凭证技术，使得服务器在不了解监护人具体身份的情况下仍能验证份额的有效性；（3）继承触发后恢复的资产以明文形式返回给继承人，未来可引入“渐进式解密”机制——继承人需要完成额外的身份认证步骤（如生物特征或多因素认证）后才能逐条解密和查看资产内容，防止继承人的设备被盗后资产直接暴露。

3.7.6 区块链层级的安全考量

智能合约层的安全分析需要单独讨论。Solidity 0.8.x 版本的内置溢出检查消除了经典的整数溢出漏洞，但合约仍面临以下安全考量：（1）重入攻击——当前合约的函数在执行外部调用之前已完成所有状态变更（checks-effects-interactions 模式），不涉及向外部合约发送 ETH，因此重入风险较低；（2）前端运行——在公开链上部署时，createPlan 和 submitShare 交易在内存池中可见，恶意参与者可能通过提高 gas 价格抢先提交。可通过 commit-reveal 方案缓解——先提交承诺哈希，在后续交易中揭示真实数据；（3）gas 消耗——createPlan 函数中的循环遍历 guardianAddresses 数组，当监护人数量较大时（如 100 人），单次交易 gas 消耗可能接近区块 gas 上限。解决方案是分批初始化监护人或使用更紧凑的存储布局；（4）升级性——当前合约不支持升级，未来可使用 UUPS 或 Transparent 代理模式引入可升级性，同时通过时间锁和治理机制防止恶意升级。这些考量在当前竞赛原型阶段不是首要问题，但在生产化部署前需进行完整的安全审计和形式化验证。

第四章 创新性说明

4.1 场景创新：数字遗产全生命周期模型

目前秘密共享、门限签名等密码学技术大多以密钥备份、企业容灾或区块链钱包恢复作为典型应用场景，研究重点通常集中于算法正确性和安全性验证。然而数字遗产继承场景同时涉及密码学、安全工程、法律规则以及社会关系管理等多个维度，其复杂程度远高于传统密钥备份问题。

本作品将门限密码学应用于数字遗产继承领域，构建了从资产托管、继承触发到秘密恢复的完整业务闭环。与传统密码保险箱不同，本系统不仅关注秘密的保存，更关注秘密在未来特定条件下的安全释放。系统需要同时解决“提前泄露风险”和“永久丢失风险”之间的矛盾——资产既不能在继承前被非法获取，也不能因持有人失联而永久不可访问。

具体而言，本作品将数字遗产继承抽象为五个有序阶段：资产托管（加密存储）→ 条件触发（时间或共识）→ 份额验证（三层密码学验证）→ 密钥恢复（拉格朗日插值）→ 资产释放（AES 解密交付）。每个阶段均有明确的输入约束和输出保证，构成了一个可审计、可复现的安全流程。

此外，本作品支持加密货币私钥、数字版权文件、云存储账号以及智能合约权限等多种数字资产类型，使门限密码学从理论验证走向实际业务应用，具有较强的现实意义和推广价值。

4.2 技术创新：双层可验证秘密共享机制

在技术实现层面，本作品的核心创新在于构建了一个三层可验证秘密共享验证管道。

传统秘密共享方案通常只保存份额数据本身，缺乏密码学强度的份额验证机制。当攻击者伪造份额或监护人提交错误数据时，系统往往只能在恢复阶段（即成功收集到足够份额后尝试恢复秘密时）才能发现问题。这种“恢复后发现错误”的模式存在严重缺陷：如果错误份额参与了拉格朗日插值，恢复出的秘密将是错误的；如果错误份额数量足够多，可能永远无法找到正确的份额组合。

本作品针对这一问题，设计了“恢复前验证”的安全模式：

1. 第一层——份额承诺验证：利用椭圆曲线离散对数问题的单向性，公开存储 $Y_i =$

- $y_i G$ 而无需担心 y_i 被反推。提交份额时验证 $y'_i G \stackrel{?}{=} Y_i$ ，确保份额值未被篡改。
2. 第二层——**Pedersen** 同态承诺验证:利用椭圆曲线同态性质验证 $Y_i \stackrel{?}{=} \sum_{j=0}^{t-1} x_i^j A_j$ ，确保份额来源于原始多项式而非其他秘密共享实例。这一层解决了跨实例份额注入攻击。
 3. 第三层——**Schnorr** 零知识证明（未来扩展）：利用 Fiat-Shamir 变换实现非交互式知识证明验证 $zG \stackrel{?}{=} R + eY_i$ ，确保提交者确实知道 y_i 而非仅仅获取了公开的 Y_i 值。这一层解决了份额承诺值窃取重用攻击。

三层机制在普通 Web 应用原型中实现了学术界论文级别的密码学安全保证。实验表明，即使攻击者仅修改份额值中的一个十六进制字符，验证流程也能够第一层准确识别异常并拒绝恢复请求。

4.3 工程创新：全栈安全工程实践

与多数密码学课程实验仅展示算法运行结果不同，本作品采用完整的全栈工程架构实现，从用户界面到密码学模块均具备独立运行和测试能力。

在系统架构设计上，本作品将密码学模块（crypto/shamir.ts）、业务逻辑模块（services/legacyPlanService.ts）、用户管理模块（services/userService.ts）、AI 服务模块（services/aiService.ts）、邮件服务模块（services/emailService.ts）和展示模块（React 前端）进行解耦，实现较好的模块化设计。密码学组件不依赖 Express、不依赖文件系统、不依赖任何业务逻辑——它可以被独立引入、独立测试，并迁移到密钥托管、企业容灾恢复等其他安全系统中。

项目同时提供完整的需求分析、系统设计、测试验证和安全分析文档，使系统不仅能够运行，而且具备较好的可复现性和可审计性，更符合实际安全产品开发流程。

在可用性工程方面，本作品设计了多步骤创建向导和自然语言 AI 助手两种交互模式，降低了非专业用户配置 Shamir 参数（ t 和 n ）的学习门槛。监护人操作被精简为“输入三个字段 + 点击提交”的极简流程，不需要理解椭圆曲线或零知识证明。

4.4 可扩展性创新

为了保证系统具备长期演进能力，本作品在架构设计阶段预留了多种扩展接口和升级路径。

在区块链层面，当前系统采用智能合约描述继承状态管理逻辑，合约中的 createPlan 和 submitShare 函数接受 bytes32 类型的承诺值，与后端的椭圆曲线承诺（压缩编码为 33 字节的十六进制字符串）兼容。未来可将完整的 Pedersen 同态承诺和 Schnorr 证明材料也提交至链上，实现完全去中心化的份额验证。

在存储层面，系统当前支持本地 JSON 文件存储，后续可通过将 encryptedAssets 字段迁移至 IPFS 或 Arweave 等去中心化存储网络（仅在数据库中保存内容标识符

CID)，实现遗产数据的长期不可篡改保存。

在业务层面，系统预留了短信通知、多因素认证、数字身份认证以及法律公证接口等扩展点。邮件服务的接口设计（emailService.sendXxx 方法）不依赖特定的邮件基础设施，可替换为任何支持 SMTP 或 API 方式的服务提供商。

在密码学层面，系统当前使用 secp256k1 椭圆曲线（与比特币和以太坊兼容），但 EC 抽象层的设计（elliptic 库的使用方式）使得切换到其他曲线（如 ed25519 或 BLS12-381）成为可能，仅需更换 EC 构造函数参数和调整域参数。

在智能化方向，当前 AI 助手基于正则规则匹配，未来可接入大语言模型（LLM）实现更灵活的自然语言理解和遗产配置建议、风险评估、继承规则自动生成以及异常行为检测等高级功能。

4.5 与现有方案的详细对比

为了更清晰地展现本作品的创新价值，下表从多个维度将本作品与现有典型方案进行系统对比：

表 4.1：本作品与现有方案的多维度对比

维度	传统遗嘱公证	中心化托管	链上多签钱包	硬件恢复服务	本作品
隐私保护	差（公证人可见）	差（平台可见）	中（链上公开）	中（厂商可见）	优（份额分散，服务端无明文）
抗单点失效	差	差（依赖平台）	优	中	优（t-of-n 门限恢复）
份额可验证	无	无	中	中	优（三层密码学验证）
资产覆盖面	窄（文书提及）	窄（平台资产）	窄（链上资产）	窄（特定硬件）	广（加密货币/云/文件/合约）
触发灵活性	差（遗嘱执行）	差（平台政策）	中（多签逻辑）	差	优（共识/时间锁/混合）
使用门槛	高（法律流程）	低	高（技术）	中	低（向导 + AI 助手）
去中心化	无	无	高	无	中（合约可选部署）
可审计性	中	低	高	低	优（全生命周期日志）

从上表可以看出，本作品在隐私保护、份额可验证性、资产覆盖面和触发灵活性等维度上具有综合优势，同时通过向导式界面和 AI 助手降低了非专业用户的使用门槛。

4.6 工程实现的创新价值

本作品的工程实现本身也构成创新贡献。当前多数密码学课程实验停留在命令行或 Jupyter Notebook 层面的算法演示，缺乏完整的交互界面、错误处理和生命周期管理。本作品将 Shamir SSS、Pedersen 承诺和 Schnorr NIZK（未来）三项密码学技术集成到一个具有生产级代码质量的全栈 Web 应用中，证明了这些技术在真实网络环境中的可行性。

具体而言，密码学模块的设计遵循以下工程原则：一是接口隔离——ShamirSecret-Sharing 接口定义清晰的 split/combine/verify 方法签名，调用方无需了解有限域运算或椭圆曲线细节；二是安全默认——构造函数自动校验域阶与群阶一致性，防止错误域参数导致密码学失效；三是优雅降级——verifyShareProof 在缺少扩展证明材料时自动回退到基础承诺验证，保证向后兼容性。

这些工程实践使得密码学模块可以被其他项目直接复用。例如，一个企业密钥托管系统可以引入 shamir.ts 而无需修改任何代码，仅需按照接口契约提供参数即可实现密钥的门限拆分和可验证恢复。

4.7 社会价值与应用前景展望

数字遗产问题不仅是一个技术问题，更是一个日益紧迫的社会问题。据统计，全球因私钥丢失而永久无法访问的比特币已超过 370 万枚（价值超过 1000 亿美元）。随着 Web3、DeFi 和 NFT 的普及，越来越多的个人财富以纯数字形式存在，而相应的继承基础设施严重滞后。本作品的技术方案为社会提供了一个可行的解决路径：通过门限密码学将“单人掌握密钥”转化为“多人协作恢复”，在隐私保护和继承可达性之间取得了工程平衡。

在应用前景方面，本作品可向以下方向演进：与法律科技（LegalTech）公司合作，将数字遗产计划与法律遗嘱签署流程整合；与加密货币交易所和钱包服务商合作，作为其“遗产托管”功能的开源参考实现；与云服务商（如 AWS KMS、Google Cloud KMS）集成，利用硬件安全模块保护份额分发过程；扩展到去中心化身份（DID）和可验证凭证（VC）领域，实现数字身份的跨代传递。

第五章 总结

5.1 工作总结

本作品数字遗产管家（Digital Legacy Guardian）围绕数字资产继承中的单点失效、隐私泄露、提前释放和伪造份额四大核心问题，完成了一个从理论到实现的全栈可运行原型系统。系统将 Shamir 秘密共享、Feldman 可验证秘密共享、Schnorr 零知识证明（未来扩展）、AES 对称加密、REST API 服务、React 前端向导、Solidity 智能合约状态机和自然语言 AI 助手等多项技术有机结合起来，实现了从资产托管到继承恢复的完整安全闭环。

在密码学层面，本作品在普通 Web 应用原型中成功落地了可验证秘密共享机制。三层验证管道——份额承诺验证、Pedersen 同态承诺验证和 Schnorr 证明（未来扩展）验证——确保份额的真实性可以在恢复操作之前得到密码学强度的验证，而非依赖“恢复后才发现错误”的脆弱模式。Fiat-Shamir 变换的使用使得零知识证明无需交互即可完成，适合 Web 应用的非持续性连接场景。

在安全工程层面，本作品贯彻了“最小化存储敏感数据”的原则。StoredShare 与 Share 的差分设计使得持久化存储中不含份额明文值；主密钥仅在创建时的局部变量中存在，拆分后即被丢弃；加密资产包的密钥本身也通过门限拆分保护。这些设计确保了即使攻击者获得服务器文件系统的完全读取权限，也无法直接获取资产内容。

在架构层面，本作品实现了密码学模块、业务逻辑模块、数据管理模块和展示模块的解耦设计。密码学模块可独立测试和迁移复用，业务服务关注生命周期管理而不涉及底层密码学细节，前端提供可视化的多步骤向导降低用户使用门槛。智能合约表达了去中心化条件下的状态约束和触发条件，为未来的链上部署提供了清晰的接口规范。

本报告依据竞赛要求编写，保留摘要、作品概述、设计与实现、测试与分析、创新性说明、总结和参考文献的标准结构，补充了大量架构图、流程图、状态图、表格和代码片段。报告避免出现学校、院系、指导教师等身份信息，满足网评匿名性要求。

5.2 不足与展望

当前系统仍属于竞赛原型阶段，在迈向生产级部署的过程中还存在以下不足：

1. 持久化方案简化：当前使用本地 JSON 文件存储，缺少事务支持、并发控制和故

障恢复能力。生产环境应将 plans 和 requests 迁移至 PostgreSQL 关系数据库，将 encryptedAssets 字段迁移至 IPFS 或 AWS S3 对象存储（数据库中仅保存 CID 或对象键）。

2. 份额分发渠道单一：当前份额仅通过电子邮件发送给监护人。虽然 SMTP 通常运行在 TLS 之上，但邮件服务提供商可以读取邮件内容。未来可引入端到端加密（使用监护人的公钥加密份额）、安全消息应用（如 Signal）或硬件安全模块（HSM）作为额外的份额分发渠道。
3. 智能合约仅本地测试：当前合约在 Hardhat 本地网络测试，尚未部署到任何公共测试网或主网。未来的工作包括：将合约部署至以太坊 Sepolia 测试网或 Layer 2 网络（如 Polygon、Arbitrum）；引入门限签名（如 BLS 门限签名）实现在链上聚合签名以完成资产转移；将 Pedersen 同态承诺材料和 Schnorr 证明的哈希提交至链上，实现链上可验证的份额提交。
4. 缺少自动化 CI/CD：当前项目缺少自动化测试套件、持续集成流水线和代码覆盖率报告。未来可引入 GitHub Actions、Jest 单元测试和端到端测试框架（如 Playwright），确保每次代码变更都经过完整的回归测试。
5. 性能未充分压测：当前原型仅在单一开发者机器上进行功能验证，未进行并发压力测试和性能基准测试。生产环境需评估在数百个并发计划下的 JSON 文件读写性能，以及门限参数较大（如 50-of-100）时的多项式运算性能。
6. 身份认证与授权不完整：当前缺少 JWT 令牌机制或 OAuth 集成，API 请求未携带认证令牌（仅通过请求参数传递 userId）。生产环境应在登录后签发 JWT，在 API 网关层进行令牌验证。
7. AI 助手能力有限：当前 AI 助手基于正则规则匹配，仅能处理预定义的指令模式。未来可集成 LLM 实现非固定模式的自然语言理解，支持更灵活的多轮对话、上下文感知和复杂参数解析。
8. 隐私保护可增强：当前计划元信息（资产名称、监护人姓名和邮箱）以明文存储。未来可考虑对敏感元数据进行加密，或采用隐私保护技术（如同态加密、安全多方计算）使得即使服务器也无法读取计划的完整元信息。

尽管存在上述不足，本作品在当前竞赛原型阶段已经实现了一个功能完整、密码学机制正确、安全设计合理的数字遗产继承系统。系统各模块的接口清晰、代码可读性良好、测试覆盖充分，为后续的工程化迭代和学术研究提供了扎实的基础。

5.3 竞赛演示方案

本作品为竞赛现场设计了完整的端到端演示流程，覆盖正常路径和多种安全异常路径，能够在有限时间内充分展示系统的功能完整性和安全防护能力。

5.3.1 演示场景一：正常创建与继承流程

该场景展示从资产持有人创建遗产计划到继承人成功恢复资产的完整过程。操作步骤如下：（1）资产持有人 Alice 注册账号并登录系统；（2）通过四步创建向导配置遗产计划——添加比特币私钥和云存储账号两项资产，设置 Bob、Carol、David 三人为监护人，配置 2-of-3 门限，选择社会共识触发模式；（3）系统完成秘密共享拆分并向三位监护人发送份额邮件；（4）Bob 和 Carol 分别登录监护人门户，输入各自的计划 ID、监护人 ID 和份额值；（5）系统对两份份额执行三层密码学验证，验证通过后达到门限，自动恢复主密钥并解密资产；（6）继承人 Eve 在继承页面查看到恢复后的完整资产信息。整个流程直观展示了门限密码学在实际 Web 应用中的集成效果。

5.3.2 演示场景二：伪造份额拒绝

该场景展示系统的安全防护能力。在场景一的基础上，Bob 尝试修改份额值中的一个十六进制字符后重新提交。系统后端执行验证管道：第一层承诺验证发现 $y'_i G \neq Y_i$ ，立即返回错误提示，拒绝该份额。该演示直观展示了三层验证机制的第一层防护——即使修改单个字符也会被密码学验证捕获。

5.3.3 演示场景三：时间锁提前触发拒绝

该场景展示时间锁模式的防护效果。Alice 创建一个 timed 模式计划，设置 30 天时间锁。Eve（继承人）立即尝试发起继承请求。系统后端计算创建时间与当前时间的差值，发现时间锁未到期，返回精确的剩余时间（如“时间锁未到期，剩余时间：30 天 0 小时 0 分钟 15 秒”），拒绝继承请求。该演示展示了系统对提前继承攻击的防护能力。

5.3.4 演示场景四：未达门限恢复拒绝

该场景展示门限控制的有效性。在 2-of-3 门限配置下，仅 Bob 一人提交有效份额。Eve 尝试调用恢复接口。系统检测到仅收集到 1 个有效份额（少于门限 2），返回“Not enough shares to recover”错误，拒绝执行恢复操作。该演示从信息论层面说明了少于 t 个份额时无法恢复秘密的数学原理在工程中的实际体现。

5.3.5 演示场景五：AI 助手引导操作

该场景展示 AI 助手降低用户使用门槛的效果。演示者通过自然语言对话界面输入：“创建一个名为我的数字遗产的遗产计划”、“添加比特币资产”、“设置监护人 Bob，邮箱 bob@example.com”、“设置门限为 2-of-3”。AI 助手逐步解析意图、提取参数、更新对话上下文，最终完成遗产计划创建。该演示展示了系统在可用性工程方面的创新——通过自然语言交互降低门限密码学参数配置的专业门槛。

以上五个演示场景覆盖了系统的核心功能路径和关键安全防护点，每个场景的演示时间约为 2-3 分钟，总演示时间控制在 15 分钟以内，适合竞赛现场的时间约束。

5.3.6 演示技术准备

为确保竞赛现场演示的顺利进行，项目做了以下技术准备：（1）监护人的份额值在计划创建时通过控制台日志输出，方便演示者分发给扮演监护人的评审专家；（2）后端服务配置为控制台日志模式，可以在终端窗口中实时展示密码学验证的计算过程，增强技术展示效果；（3）前端使用 TailwindCSS 的响应式设计，在投影仪分辨率下保持良好的可读性；（4）合约测试使用 Hardhat 的内置网络和 `evm_increaseTime` 方法，可以在几秒内模拟数天的时间流逝，高效展示时间锁机制。

5.3.7 演示脚本示例

以下为标准演示脚本的简化版，供竞赛现场参考：

开场（30 秒）：介绍数字遗产继承的现实问题——全球超过 370 万枚比特币因私钥丢失而永久不可访问，引出本作品的解决思路。

场景一（3 分钟）：Alice 创建遗产计划 → 系统自动拆分秘密并加密资产 → Bob 和 Carol 提交份额 → 系统验证通过 → 资产成功恢复并展示。此阶段强调三层验证管道的工作过程和门限恢复的数学原理。

场景二（2 分钟）：Bob 尝试提交被篡改的份额 → 系统第一层验证失败 → 返回明确错误信息。此阶段强调“恢复前验证”的安全设计哲学。

场景三（2 分钟）：展示时间锁模式 → 提前触发被拒绝 → 系统精确显示剩余时间。此阶段强调时间锁对提前继承攻击的防护。

场景四（2 分钟）：仅 1 个有效份额尝试恢复 → 系统检测未达门限 → 拒绝恢复。此阶段从信息论角度说明少于 t 个份额无法获取任何秘密信息。

场景五（2 分钟）：通过 AI 助手自然语言对话完成计划创建。此阶段展示系统在可用性工程方面的创新。

总结（30 秒）：回顾系统架构和创新点，强调密码学安全性与用户操作便利性的平衡。

参考文献

- [1] Shamir A. How to share a secret[J]. Communications of the ACM, 1979, 22(11): 612-613.
- [2] Feldman P. A practical scheme for non-interactive verifiable secret sharing[C]//Proceedings of the 28th Annual Symposium on Foundations of Computer Science. IEEE, 1987: 427-438.
- [3] Schnorr C P. Efficient signature generation by smart cards[J]. Journal of Cryptology, 1991, 4(3): 161-174.
- [4] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system[EB/OL]. 2008.
- [5] Wood G. Ethereum: A secure decentralised generalised transaction ledger[EB/OL]. 2014.
- [6] Benaloh J. Secret sharing homomorphisms: Keeping shares of a secret secret[C]//Advances in Cryptology —CRYPTO '86. Springer, 1987: 251-260.
- [7] Pedersen T P. Non-interactive and information-theoretic secure verifiable secret sharing[C]//Advances in Cryptology —CRYPTO '91. Springer, 1991: 129-140.
- [8] National Institute of Standards and Technology. Digital Signature Standard (DSS), FIPS PUB 186-5[S]. 2023.
- [9] National Institute of Standards and Technology. Recommendation for Block Cipher Modes of Operation, SP 800-38A[S]. 2001.
- [10] National Institute of Standards and Technology. Recommendation for Password-Based Key Derivation, SP 800-132[S]. 2010.
- [11] Bernstein D J, Duif N, Lange T, et al. High-speed high-security signatures[J]. Journal of Cryptographic Engineering, 2012, 2(2): 77-89.
- [12] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems[C]//Advances in Cryptology —CRYPTO '86. Springer, 1987: 186-194.
- [13] OpenZeppelin. Smart Contract Security Guidelines[EB/OL]. <https://docs.openzeppelin.com/>.
- [14] Solidity Team. Solidity v0.8 Documentation[EB/OL]. <https://docs.soliditylang.org/>.

- [15] Ethereum Foundation. Ethereum Yellow Paper[EB/OL]. 2024.
- [16] Hardhat. Ethereum development environment documentation[EB/OL]. <https://hardhat.org/docs>.
- [17] React Team. React v18 Documentation[EB/OL]. <https://react.dev/>.
- [18] Express.js. Express 4.x API Reference[EB/OL]. <https://expressjs.com/>.
- [19] TypeScript Team. TypeScript Handbook[EB/OL]. <https://www.typescriptlang.org/docs/>.
- [20] elliptic. Fast Elliptic Curve Cryptography in plain JavaScript[EB/OL]. <https://github.com/indutny/elliptic>.
- [21] CryptoJS. JavaScript library of crypto standards[EB/OL]. <https://github.com/brix/crypto-js>.
- [22] GB/T 7714-2015. 信息与文献参考文献著录规则 [S]. 中国标准出版社, 2015.
- [23] Merkle R C. A digital signature based on a conventional encryption function[C]//Advances in Cryptology —CRYPTO '87. Springer, 1987: 369-378.
- [24] Bellare M, Rogaway P. Random oracles are practical: A paradigm for designing efficient protocols[C]//Proceedings of the 1st ACM Conference on Computer and Communications Security. 1993: 62-73.
- [25] Menezes A J, Van Oorschot P C, Vanstone S A. Handbook of Applied Cryptography[M]. CRC Press, 1996.